

Student:	Email:
Amatun Haque	amatunh.student@wust.edu

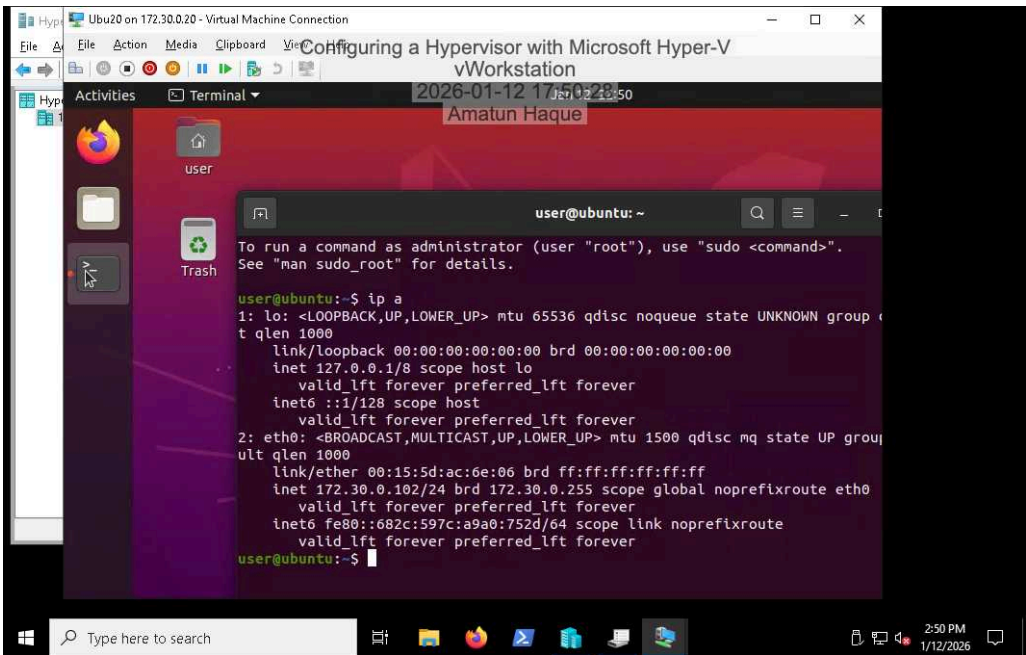
Time on Task:	Progress:
70 hours, 30 minutes	72%

Report Generated: Friday, January 16, 2026 at 1:09 PM

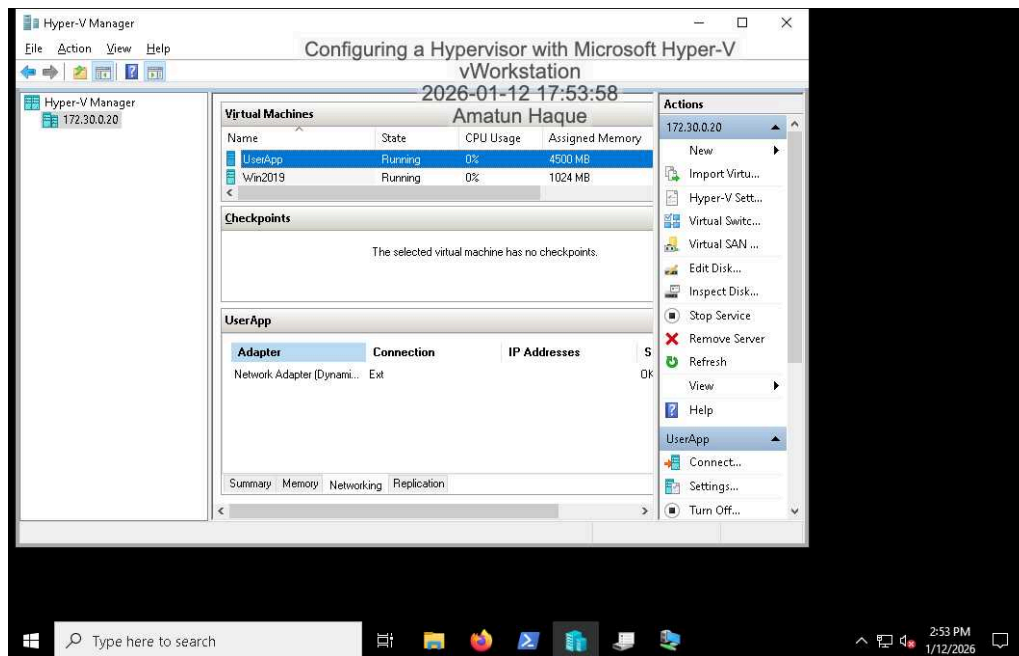
Hands-On Demonstration

Part 1: Basic Operations using Hyper-V Manager

- 22. Make a screen capture showing the network interface configuration on the Ubu20 virtual machine.

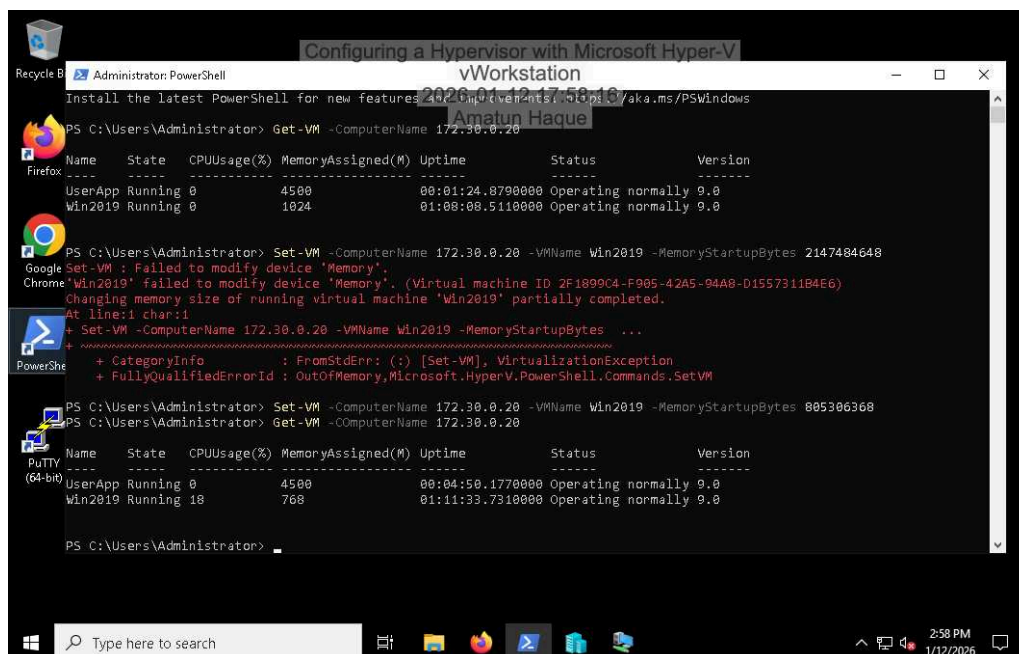


37. Make a screen capture showing the virtual machine list with the new name and memory allocation.

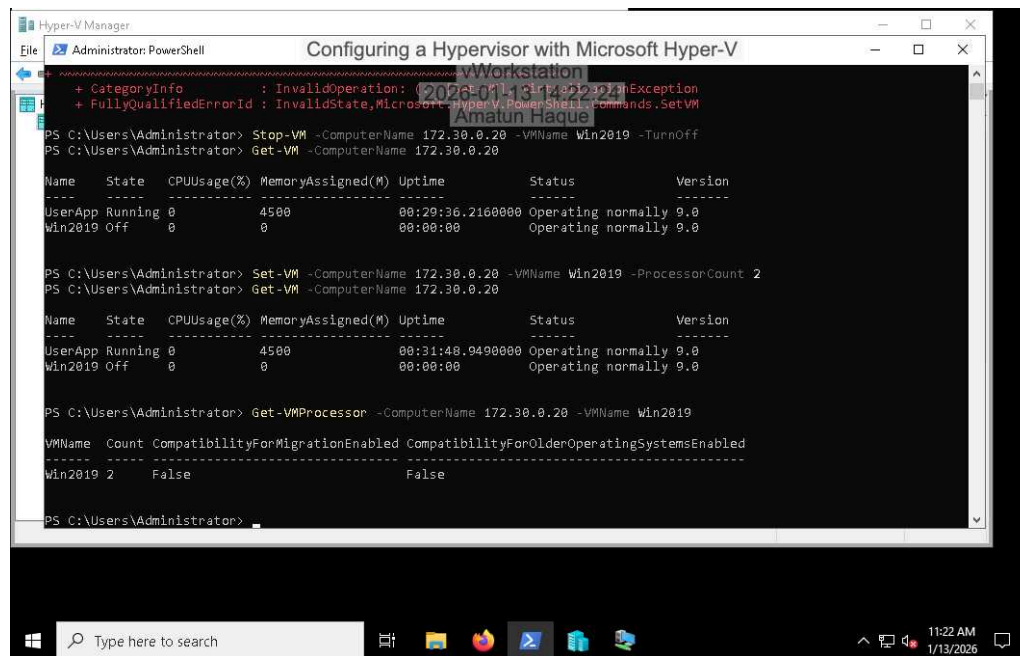


Part 2: Basic Operations Using PowerShell

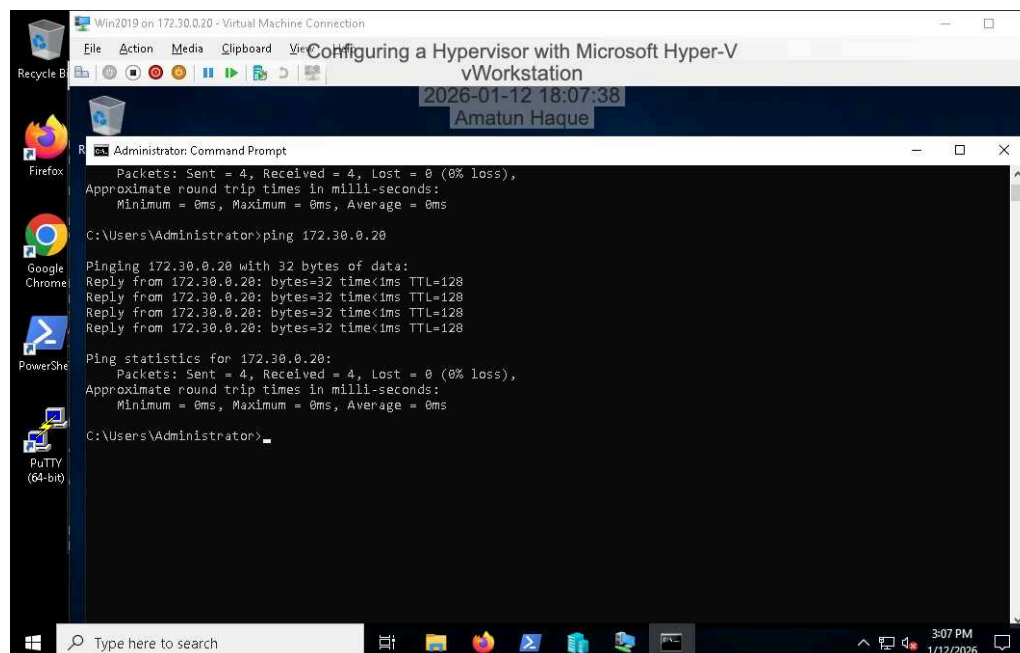
6. Make a screen capture showing the new memory allocation.



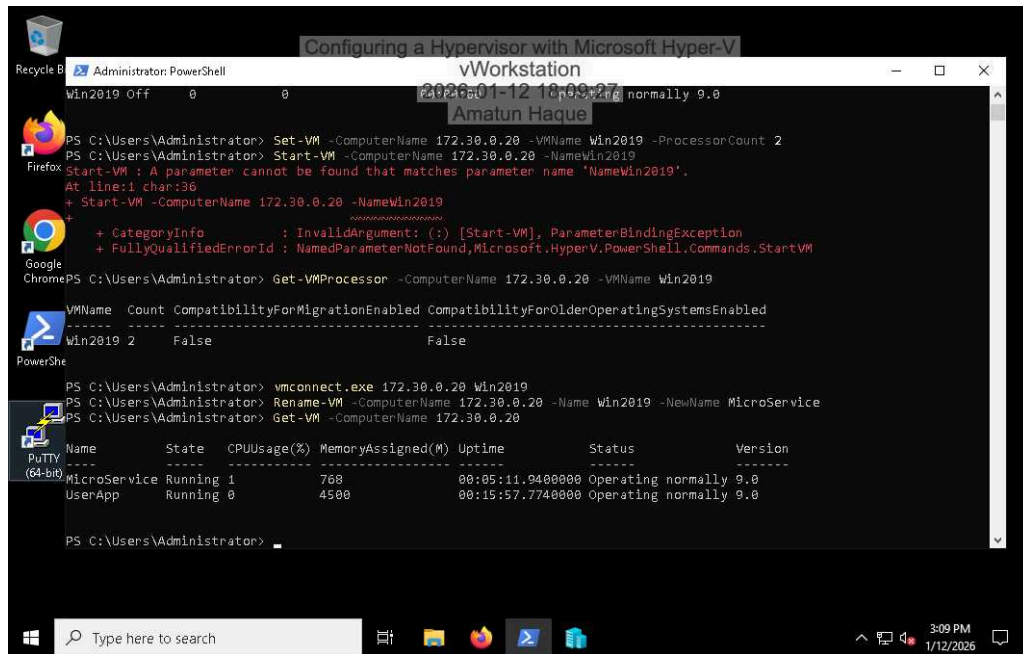
14. **Make a screen capture** showing the Get-VM output indicating the machine is using two processors.



23. **Make a screen capture** showing the successful pings.

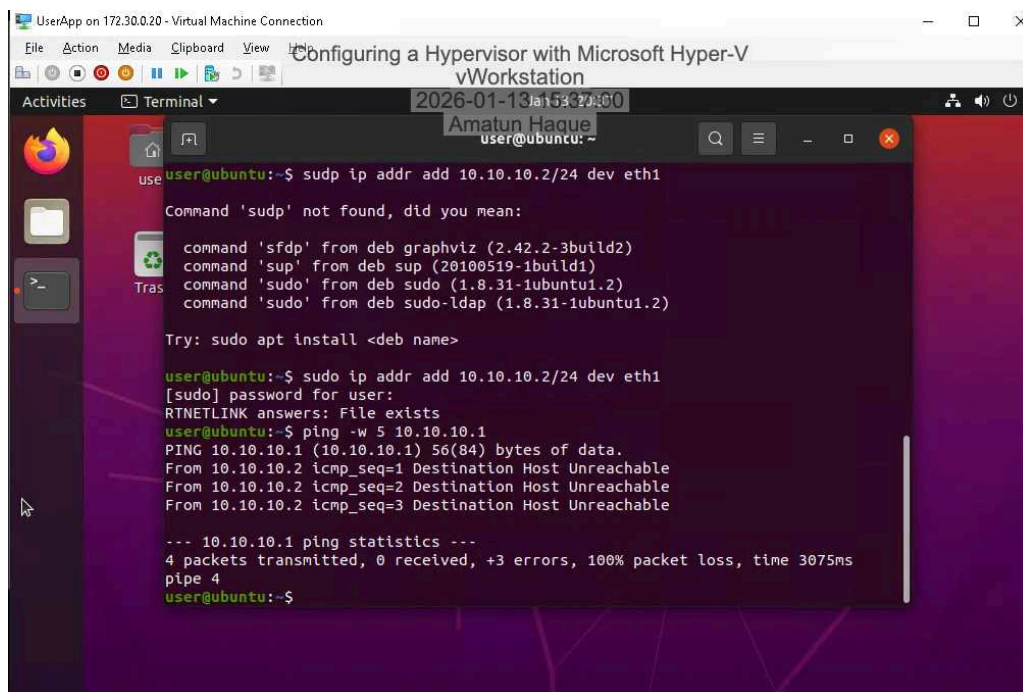


28. Make a screen capture showing the VM list with the new name.

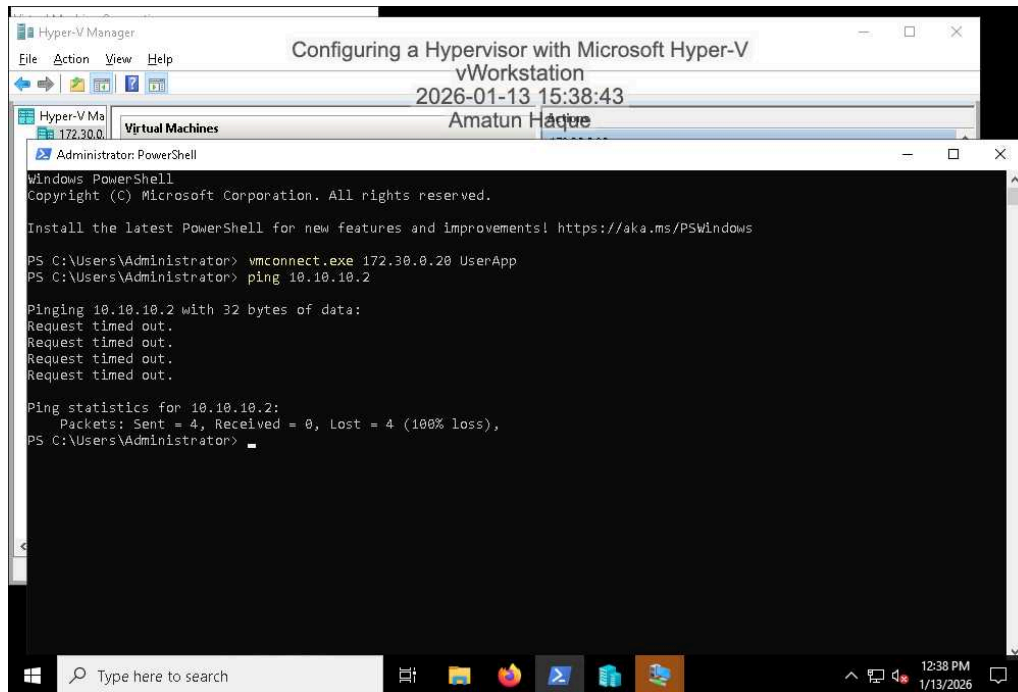


Part 3: Add a New Private Network

39. Make a screen capture showing the successful ping from the Windows VM to the Linux VM.



40. **Make a screen capture** showing the successful ping from the Linux VM to the Windows VM.



Challenge and Analysis

Part 1: Determine State of Hyper-V Integration Services

3. **Make a screen capture** showing the **state of all the MicroService VM's Integration Services** in the cmdlet output.

Incomplete

6. **Describe** what guest-to-host functionality is currently unavailable for the MicroService VM.

Incomplete

Part 2: Create Checkpoints of Hyper-V VMs

6. **Make a screen capture** showing **the snapshots for each VM**.

Incomplete

Student:	Email:
Amatun Haque	amatunh.student@wust.edu

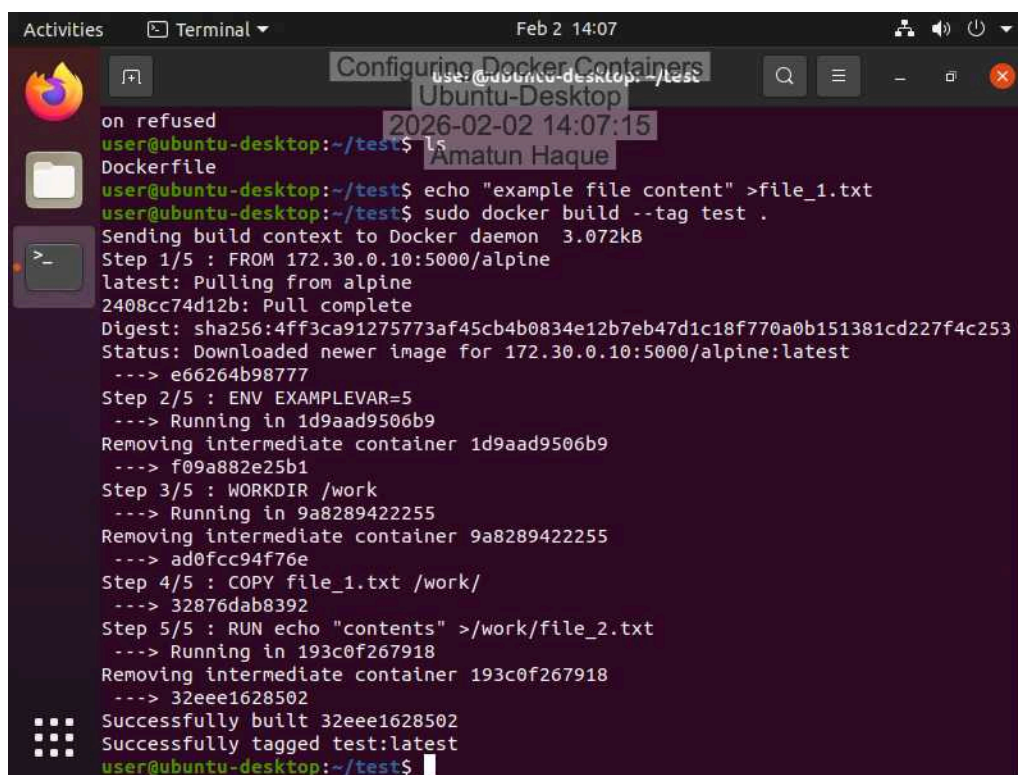
Time on Task:	Progress:
3 hours, 12 minutes	63%

Report Generated: Monday, February 2, 2026 at 2:37 PM

Hands-On Demonstration

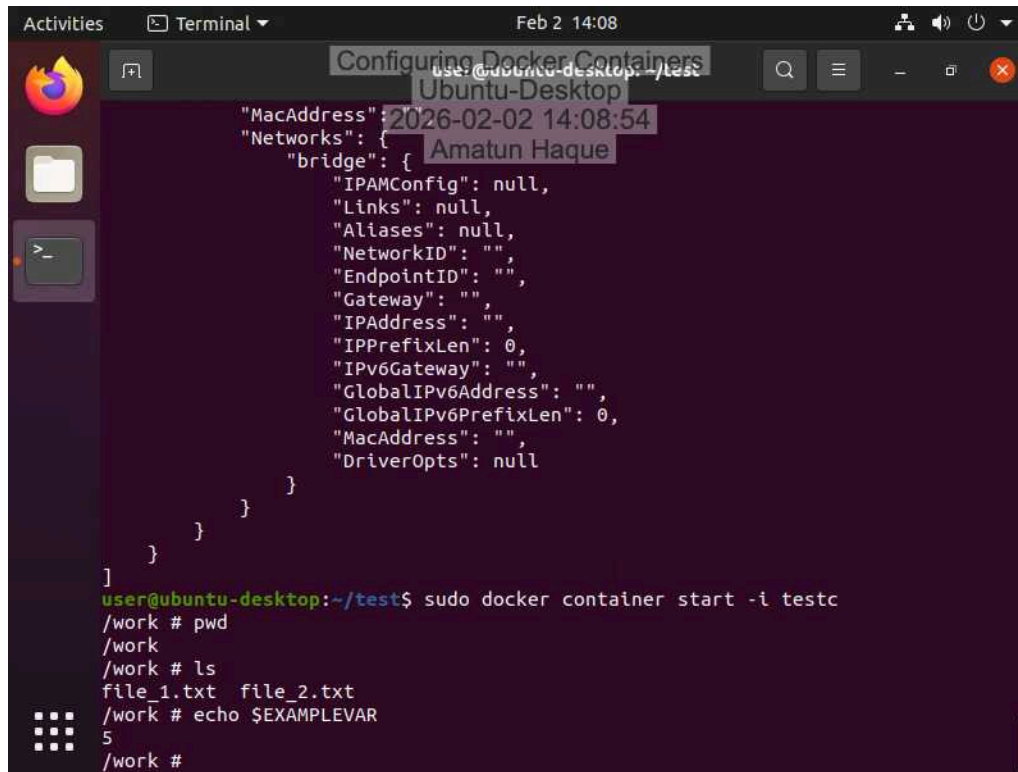
Part 1: Creating and Running a Docker Container

9. Make a screen capture showing the **successful build of the test image**.

A terminal window titled 'Terminal' with a date and time of 'Feb 2 14:07'. The terminal shows a series of commands and their outputs. The user is in a directory named '/test'. They run 'ls' and see 'Dockerfile'. Then they run 'echo "example file content" >file_1.txt'. Next, they run 'sudo docker build --tag test .' which starts a Docker build process. The build has five steps: 1. FROM 172.30.0.10:5000/alpine, 2. ENV EXAMPLEVAR=5, 3. WORKDIR /work, 4. COPY file_1.txt /work/, and 5. RUN echo "contents" >/work/file_2.txt. The build is successful, resulting in a tagged image 'test:latest'.

```
on refused
user@ubuntu-desktop:~/test$ ls
Dockerfile
user@ubuntu-desktop:~/test$ echo "example file content" >file_1.txt
user@ubuntu-desktop:~/test$ sudo docker build --tag test .
Sending build context to Docker daemon 3.072kB
Step 1/5 : FROM 172.30.0.10:5000/alpine
latest: Pulling from alpine
2408cc74d12b: Pull complete
Digest: sha256:4ff3ca91275773af45cb4b0834e12b7eb47d1c18f770a0b151381cd227f4c253
Status: Downloaded newer image for 172.30.0.10:5000/alpine:latest
--> e66264b98777
Step 2/5 : ENV EXAMPLEVAR=5
--> Running in 1d9aad9506b9
Removing intermediate container 1d9aad9506b9
--> f09a882e25b1
Step 3/5 : WORKDIR /work
--> Running in 9a8289422255
Removing intermediate container 9a8289422255
--> ad0fcc94f76e
Step 4/5 : COPY file_1.txt /work/
--> 32876dab8392
Step 5/5 : RUN echo "contents" >/work/file_2.txt
--> Running in 193c0f267918
Removing intermediate container 193c0f267918
--> 32eee1628502
Successfully built 32eee1628502
Successfully tagged test:latest
user@ubuntu-desktop:~/test$
```

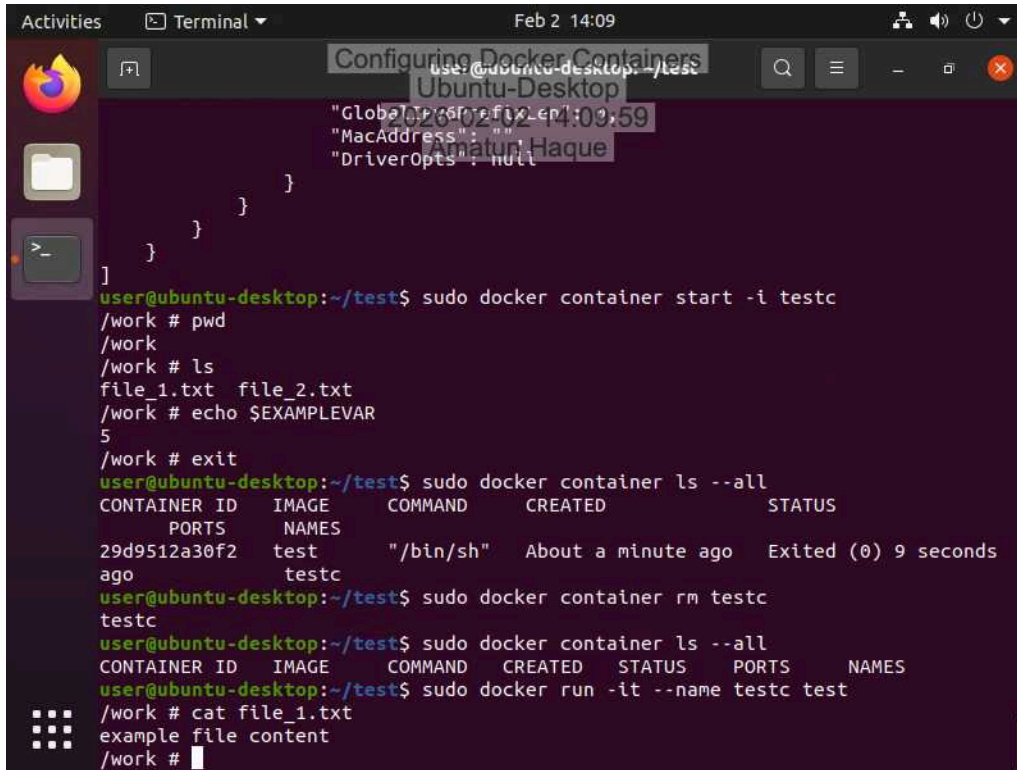
19. Make a screen capture showing the **working directory**, **file listing**, and **environment variable value**.



The screenshot shows a terminal window titled "Configuring Docker Containers" with the subtitle "Ubuntu-Desktop". The terminal displays a JSON configuration for a Docker network, followed by the command to start a container named "testc". Inside the container, the user runs "pwd", "ls", and "echo \$EXAMPLEVAR".

```
user@ubuntu-desktop: ~/test
"MacAddress": "2026-02-02 14:08:54",
"Networks": {
  "bridge": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": null,
    "NetworkID": "",
    "EndpointID": "",
    "Gateway": "",
    "IPAddress": "",
    "IPPrefixLen": 0,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "MacAddress": "",
    "DriverOpts": null
  }
}
]
user@ubuntu-desktop:~/test$ sudo docker container start -i testc
/work # pwd
/work
/work # ls
file_1.txt file_2.txt
/work # echo $EXAMPLEVAR
5
/work #
```


26. Make a screen capture showing both your docker run command and the contents of the file_1.txt.

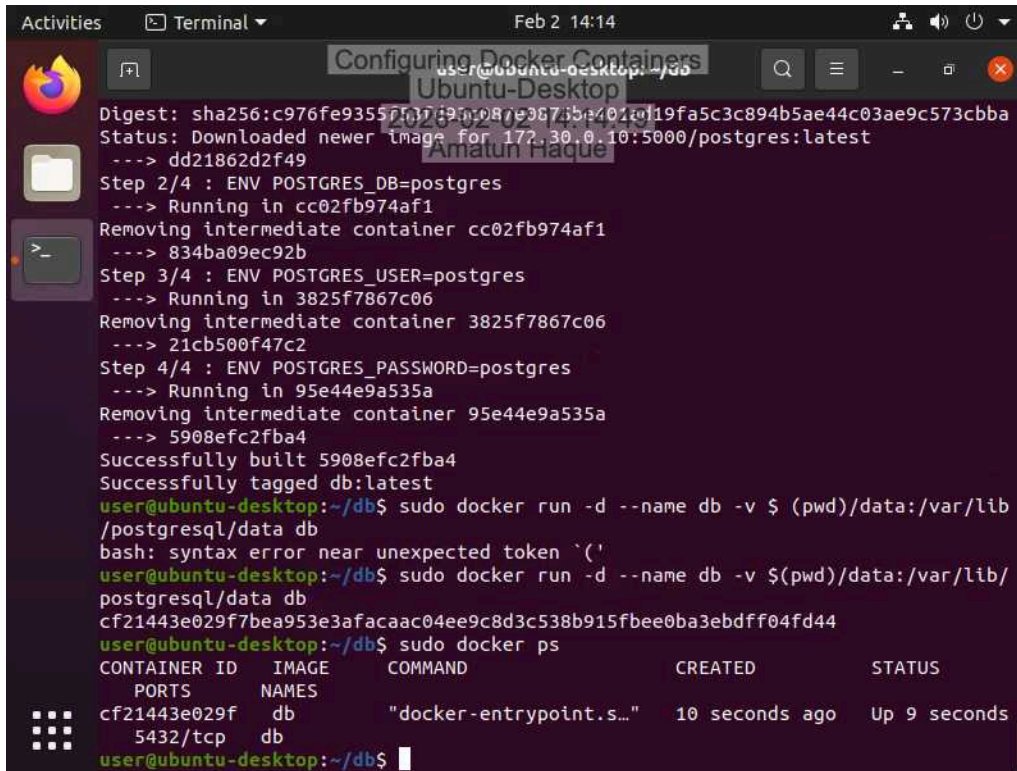


A terminal window titled "Terminal" with a dark background. The prompt is "user@ubuntu-desktop: ~/test". The terminal shows a series of commands and their outputs. First, a Docker container named "testc" is started with the command "sudo docker container start -i testc". The prompt changes to "/work #". The user runs "pwd", "ls", and "echo \$EXAMPLEVAR". The output of "ls" shows "file_1.txt" and "file_2.txt". The output of "echo" is "5". Then, the user runs "exit". Next, the user lists all containers with "sudo docker container ls --all", showing a table with columns: CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS, NAMES. The table shows one container: 29d9512a30f2, test, "/bin/sh", About a minute ago, Exited (0) 9 seconds ago, testc. Then, the user removes the container with "sudo docker container rm testc". The user lists all containers again, showing the same table. Finally, the user runs "sudo docker run -it --name testc test" and the prompt changes to "/work #". The user runs "cat file_1.txt" and the output is "example file content".

```
user@ubuntu-desktop: ~/test
}
}
}
]
user@ubuntu-desktop:~/test$ sudo docker container start -i testc
/work # pwd
/work
/work # ls
file_1.txt  file_2.txt
/work # echo $EXAMPLEVAR
5
/work # exit
user@ubuntu-desktop:~/test$ sudo docker container ls --all
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
29d9512a30f2   test     "/bin/sh"   About a minute ago   Exited (0) 9 seconds ago   testc
user@ubuntu-desktop:~/test$ sudo docker container rm testc
testc
user@ubuntu-desktop:~/test$ sudo docker container ls --all
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
user@ubuntu-desktop:~/test$ sudo docker run -it --name testc test
/work # cat file_1.txt
example file content
/work #
```

Part 2: Configuring an Application with Multiple Containers

7. Make a screen capture showing the running db container.



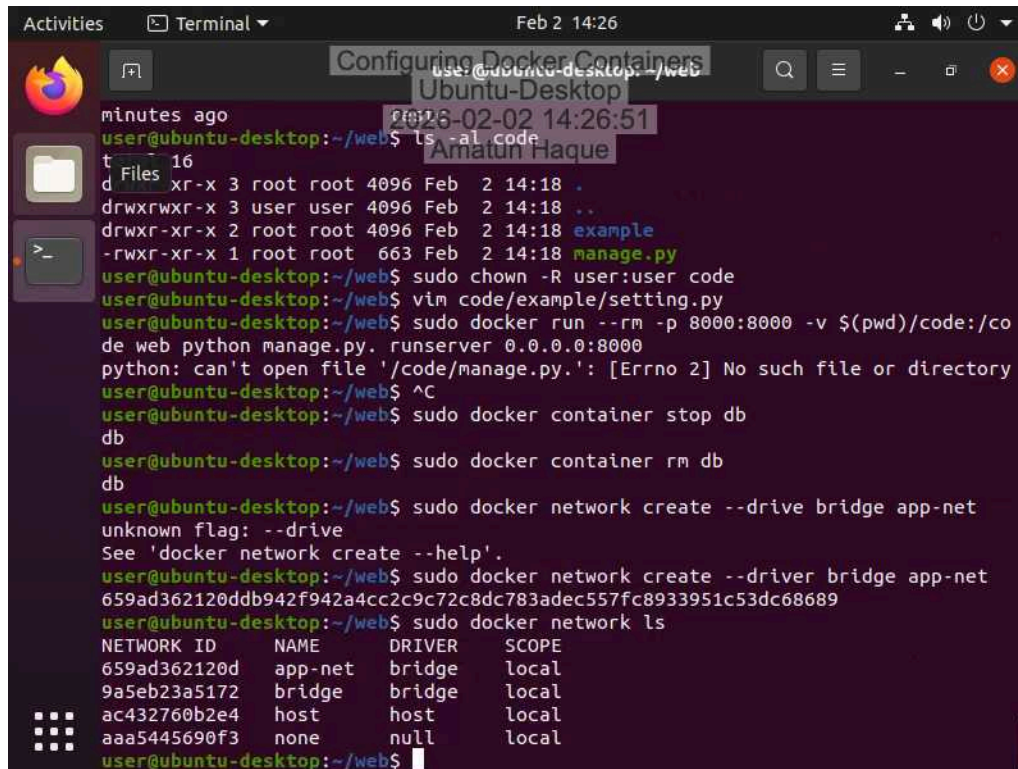
The screenshot shows a terminal window titled 'Terminal' with a date and time of 'Feb 2 14:14'. The user is at the prompt 'user@ubuntu-desktop: ~/db'. The terminal output shows the following steps:

```
Digest: sha256:c976fe93557f71391087e3675b401e019fa5c3c894b5ae44c03ae9c573cbb
Status: Downloaded newer image for 172.30.0.10:5000/postgres:latest
---> dd21862d2f49
Step 2/4 : ENV POSTGRES_DB=postgres
---> Running in cc02fb974af1
Removing intermediate container cc02fb974af1
---> 834ba09ec92b
Step 3/4 : ENV POSTGRES_USER=postgres
---> Running in 3825f7867c06
Removing intermediate container 3825f7867c06
---> 21cb500f47c2
Step 4/4 : ENV POSTGRES_PASSWORD=postgres
---> Running in 95e44e9a535a
Removing intermediate container 95e44e9a535a
---> 5908efc2fba4
Successfully built 5908efc2fba4
Successfully tagged db:latest
user@ubuntu-desktop:~/db$ sudo docker run -d --name db -v $(pwd)/data:/var/lib
/postgresql/data db
bash: syntax error near unexpected token `('
user@ubuntu-desktop:~/db$ sudo docker run -d --name db -v $(pwd)/data:/var/lib/
postgresql/data db
cf21443e029f7bea953e3afacaac04ee9c8d3c538b915fbee0ba3ebdff04fd44
user@ubuntu-desktop:~/db$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
cf21443e029f	db	"docker-entrypoint.s..."	10 seconds ago	Up 9 seconds

The terminal also shows the container's ports: 5432/tcp mapped to db.

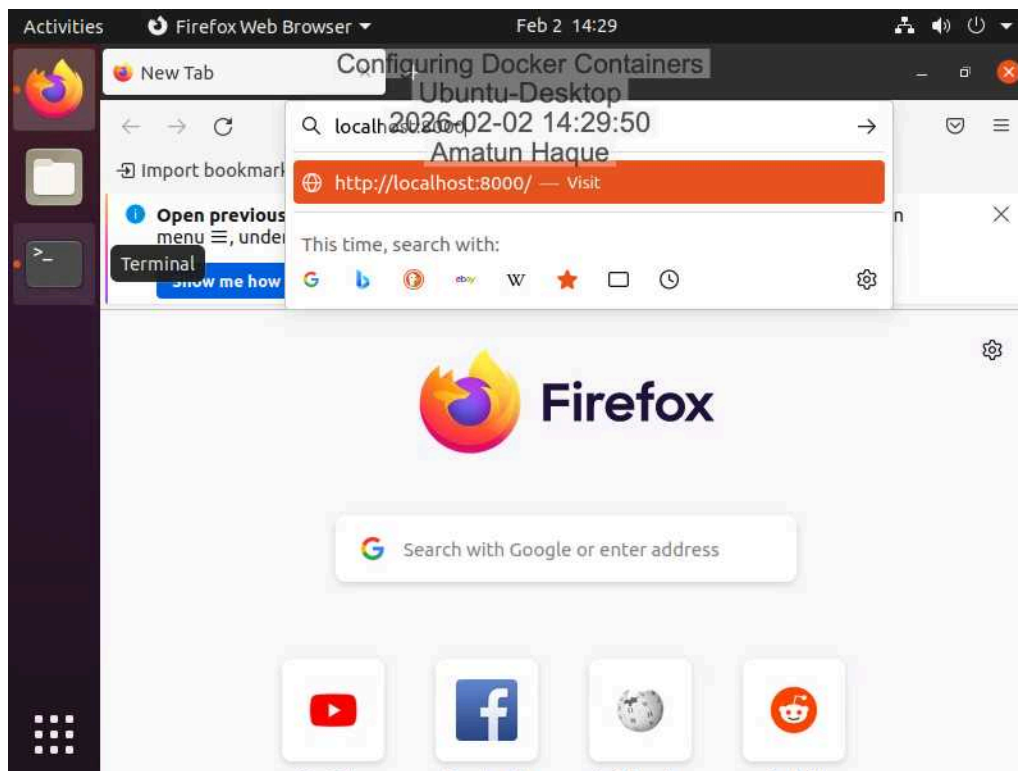
24. Make a screen capture showing the command creating the network and the resulting id.



A terminal window titled 'Configuring Docker Containers' on an Ubuntu-Desktop. The window shows a series of commands and their outputs. The user is in the directory ~/web. The commands executed are: `ls -al code`, `sudo chown -R user:user code`, `vim code/example/setting.py`, `sudo docker run --rm -p 8000:8000 -v $(pwd)/code:/code web python manage.py, runserver 0.0.0.0:8000`, `sudo docker container stop db`, `sudo docker container rm db`, and `sudo docker network create --driver bridge app-net`. The output of the last command shows the network ID 659ad362120ddb942f942a4cc2c9c72c8dc783adec557fc8933951c53dc68689. The user then runs `sudo docker network ls`, which displays a table of networks.

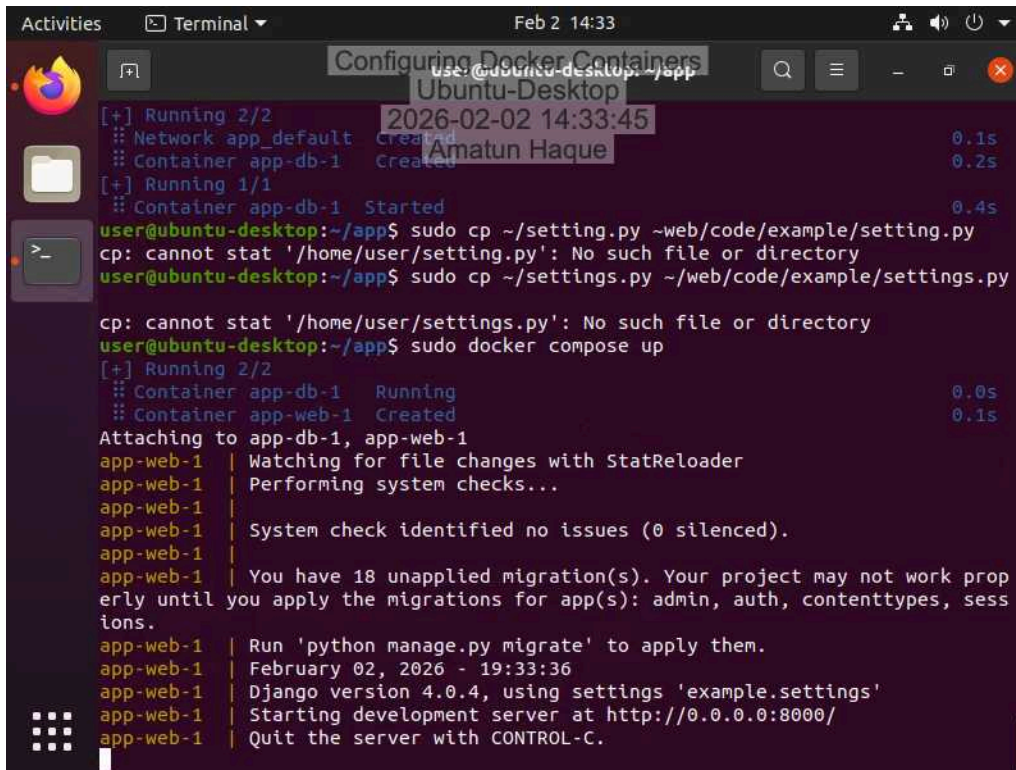
```
minutes ago
user@ubuntu-desktop:~/web$ ls -al code
total 16
drwxrwxr-x 3 root root 4096 Feb  2 14:18 .
drwxrwxr-x 3 user user 4096 Feb  2 14:18 ..
drwxr-xr-x 2 root root 4096 Feb  2 14:18 example
-rwxr-xr-x 1 root root 663 Feb  2 14:18 manage.py
user@ubuntu-desktop:~/web$ sudo chown -R user:user code
user@ubuntu-desktop:~/web$ vim code/example/setting.py
user@ubuntu-desktop:~/web$ sudo docker run --rm -p 8000:8000 -v $(pwd)/code:/code web python manage.py, runserver 0.0.0.0:8000
python: can't open file '/code/manage.py.': [Errno 2] No such file or directory
user@ubuntu-desktop:~/web$ ^C
user@ubuntu-desktop:~/web$ sudo docker container stop db
db
user@ubuntu-desktop:~/web$ sudo docker container rm db
db
user@ubuntu-desktop:~/web$ sudo docker network create --driver bridge app-net
unknown flag: --driver
See 'docker network create --help'.
user@ubuntu-desktop:~/web$ sudo docker network create --driver bridge app-net
659ad362120ddb942f942a4cc2c9c72c8dc783adec557fc8933951c53dc68689
user@ubuntu-desktop:~/web$ sudo docker network ls
NETWORK ID        NAME        DRIVER        SCOPE
659ad362120d      app-net     bridge        local
9a5eb23a5172      bridge     bridge        local
ac432760b2e4      host       host          local
aaa5445690f3      none       null          local
user@ubuntu-desktop:~/web$
```

30. Make a screen capture showing the Django page.



Part 3: Using Docker Compose

11. Make a screen capture showing output from the compose up command.

A terminal window titled 'Configuring Docker Containers' and 'Ubuntu-Desktop' showing the output of a Docker Compose command. The window has a dark background with light-colored text. The terminal shows the execution of 'docker compose up', which creates and starts containers 'app-db-1' and 'app-web-1'. It then shows the output of the 'app-web-1' container, which includes Django system checks and a message about 18 unapplied migrations. The terminal also shows the user attempting to copy files from the host to the container, which fails due to file not found errors.

```
Activities Terminal Feb 2 14:33
Configuring Docker Containers
Ubuntu-Desktop
2026-02-02 14:33:45
Amatun Haque
[+] Running 2/2
:: Network app_default Created 0.1s
:: Container app-db-1 Created 0.2s
[+] Running 1/1
:: Container app-db-1 Started 0.4s
user@ubuntu-desktop:~/app$ sudo cp ~/setting.py ~/web/code/example/setting.py
cp: cannot stat '/home/user/setting.py': No such file or directory
user@ubuntu-desktop:~/app$ sudo cp ~/settings.py ~/web/code/example/settings.py
cp: cannot stat '/home/user/settings.py': No such file or directory
user@ubuntu-desktop:~/app$ sudo docker compose up
[+] Running 2/2
:: Container app-db-1 Running 0.0s
:: Container app-web-1 Created 0.1s
Attaching to app-db-1, app-web-1
app-web-1 | Watching for file changes with StatReloader
app-web-1 | Performing system checks...
app-web-1 |
app-web-1 | System check identified no issues (0 silenced).
app-web-1 |
app-web-1 | You have 18 unapplied migration(s). Your project may not work prop
erly until you apply the migrations for app(s): admin, auth, contenttypes, sess
ions.
app-web-1 | Run 'python manage.py migrate' to apply them.
app-web-1 | February 02, 2026 - 19:33:36
app-web-1 | Django version 4.0.4, using settings 'example.settings'
app-web-1 | Starting development server at http://0.0.0.0:8000/
app-web-1 | Quit the server with CONTROL-C.
```

Challenge and Analysis

Part 1: Create an Image to Serve a Static File

Make a screen capture showing the **successful build of the web2 image**.

Incomplete

Make a screen capture showing the **directory listing served on port 8001**.

Incomplete

Part 2: Add the Static Service to the Docker Compose File

Make a screen capture showing the **creation of the web, web2, and db containers** in the docker compose output.

Incomplete

Make a screen capture showing the **directory listing served on port 8080**.

Incomplete

Student:	Email:
Amatun Haque	amatunh.student@wust.edu

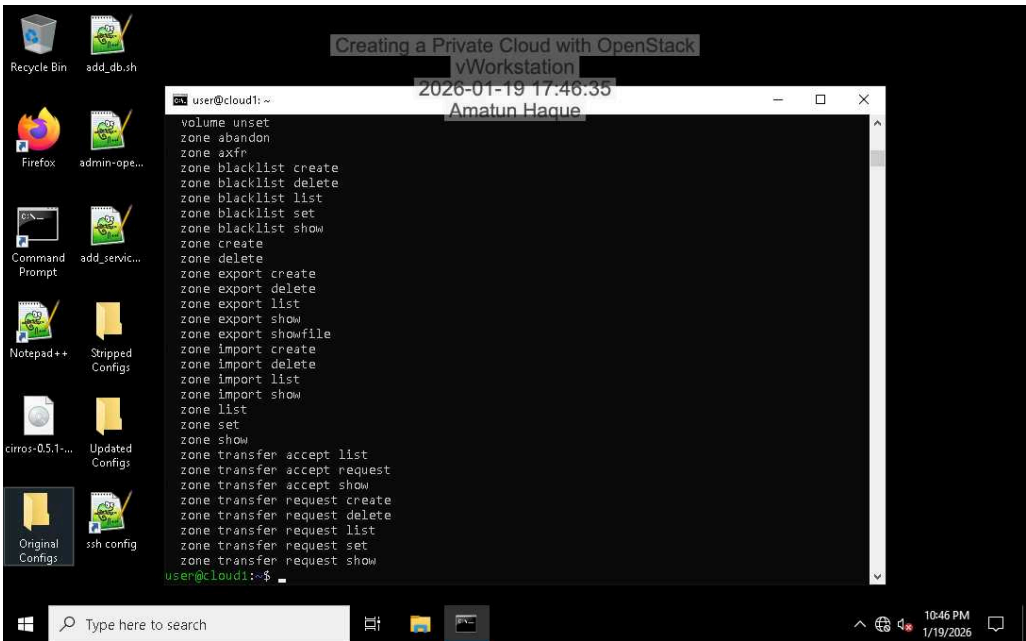
Time on Task:	Progress:
32 hours, 6 minutes	71%

Report Generated: Tuesday, January 20, 2026 at 9:14 PM

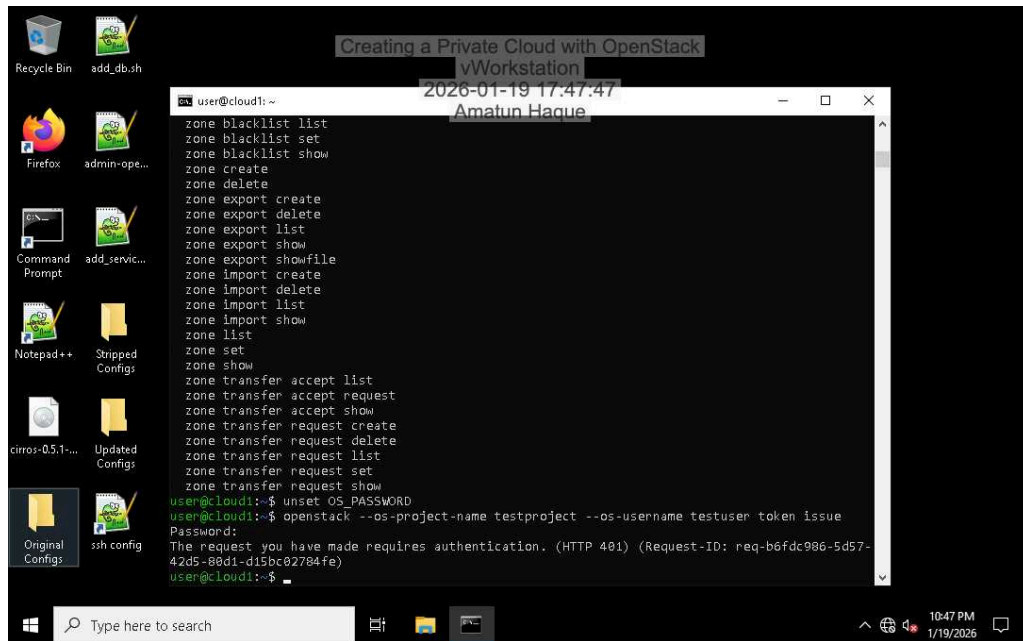
Hands-On Demonstration

Part 1: Set Up Basic Services on the Controller Node

47. Make a screen capture showing the information about the test project, test user, and test role.



51. Make a screen capture showing the generated token information.



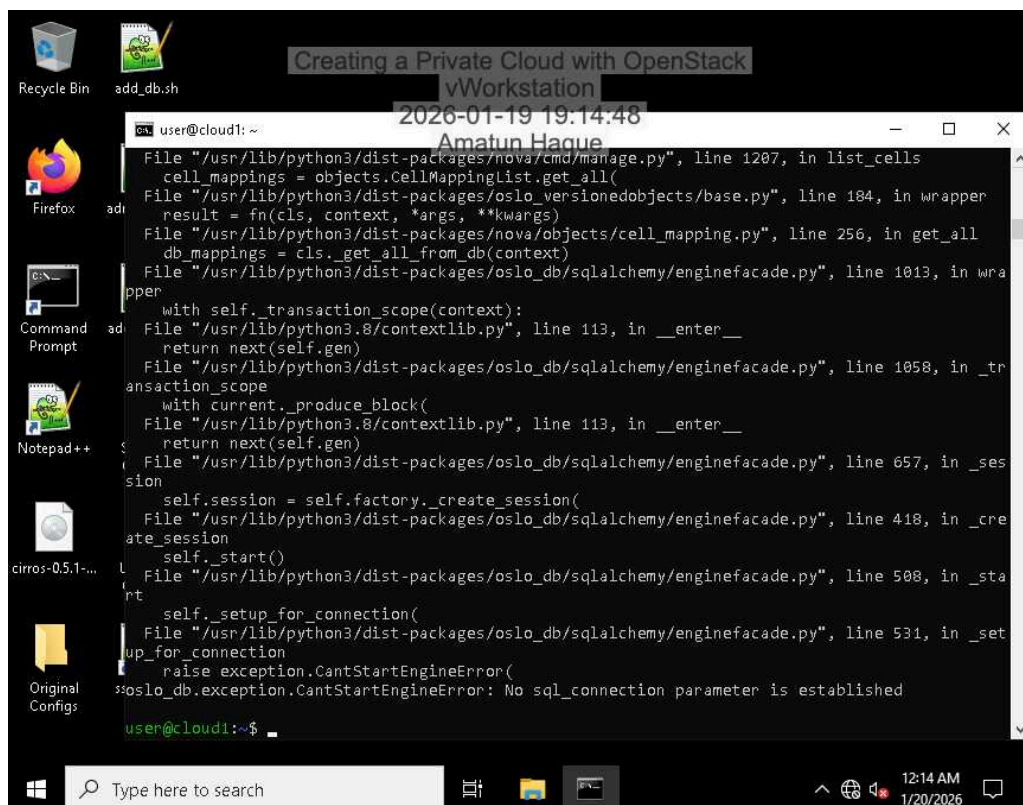
The screenshot shows a terminal window titled "Creating a Private Cloud with OpenStack" with a subtitle "vWorkstation" and "Amatun Haque". The terminal output lists various OpenStack CLI commands for zone management. The user then runs the command to issue a token for a test user.

```
user@cloud1:~$ zone blacklist list
zone blacklist set
zone blacklist show
zone create
zone delete
zone export create
zone export delete
zone export list
zone export show
zone export showfile
zone import create
zone import delete
zone import list
zone import show
zone list
zone set
zone show
zone transfer accept list
zone transfer accept request
zone transfer accept show
zone transfer request create
zone transfer request delete
zone transfer request list
zone transfer request set
zone transfer request show

user@cloud1:~$ unset OS_PASSWORD
user@cloud1:~$ openstack --os-project-name testproject --os-username testuser token issue
Password:
The request you have made requires authentication. (HTTP 401) (Request-ID: req-b6fdc986-5d57-42d5-88d1-d15bc02784fe)
user@cloud1:~$
```

Part 2: Set Up the Compute Service

22. Make a screen capture showing cell0 and cell1 in the command output.



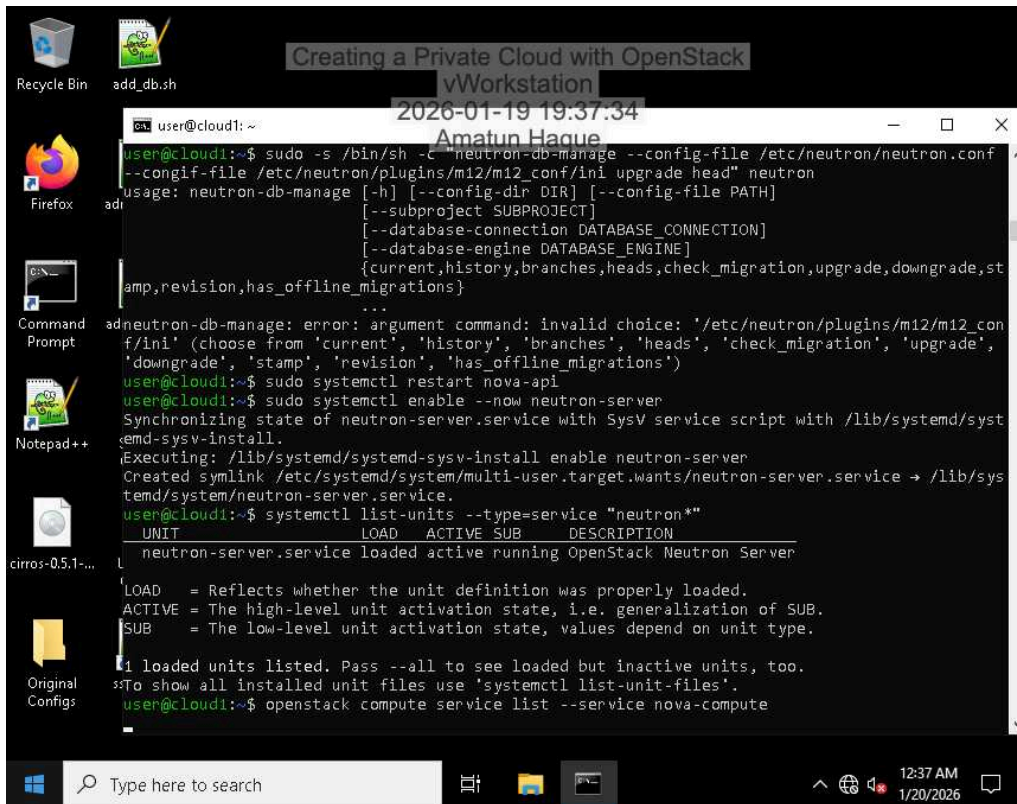
The screenshot shows a terminal window titled "Creating a Private Cloud with OpenStack" with a subtitle "vWorkstation" and "Amatun Haque". The terminal output shows a traceback of a Python exception, indicating a failure to establish a database connection.

```
File "/usr/lib/python3/dist-packages/nova/cmd/manage.py", line 1207, in list_cells
cell_mappings = objects.CellMappingList.get_all(
File "/usr/lib/python3/dist-packages/oslo_versionedobjects/base.py", line 184, in wrapper
result = fn(cls, context, *args, **kwargs)
File "/usr/lib/python3/dist-packages/nova/objects/cell_mapping.py", line 256, in get_all
db_mappings = cls._get_all_from_db(context)
File "/usr/lib/python3/dist-packages/oslo_db/sqlalchemy/enginefacade.py", line 1013, in wrapper
with self.transaction_scope(context):
File "/usr/lib/python3.8/contextlib.py", line 113, in __enter__
return next(self.gen)
File "/usr/lib/python3/dist-packages/oslo_db/sqlalchemy/enginefacade.py", line 1058, in transaction_scope
with current._produce_block(
File "/usr/lib/python3.8/contextlib.py", line 113, in __enter__
return next(self.gen)
File "/usr/lib/python3/dist-packages/oslo_db/sqlalchemy/enginefacade.py", line 657, in _session
self.session = self.factory._create_session(
File "/usr/lib/python3/dist-packages/oslo_db/sqlalchemy/enginefacade.py", line 418, in _create_session
self._start()
File "/usr/lib/python3/dist-packages/oslo_db/sqlalchemy/enginefacade.py", line 508, in _start
self._setup_for_connection(
File "/usr/lib/python3/dist-packages/oslo_db/sqlalchemy/enginefacade.py", line 531, in _setup_for_connection
raise exception.CantStartEngineError(
oslo_db.exception.CantStartEngineError: No sql_connection parameter is established

user@cloud1:~$
```

Part 3: Set Up the Network Service and Web GUI

14. Make a screen capture showing that cloud2 is an active compute node for the stack.



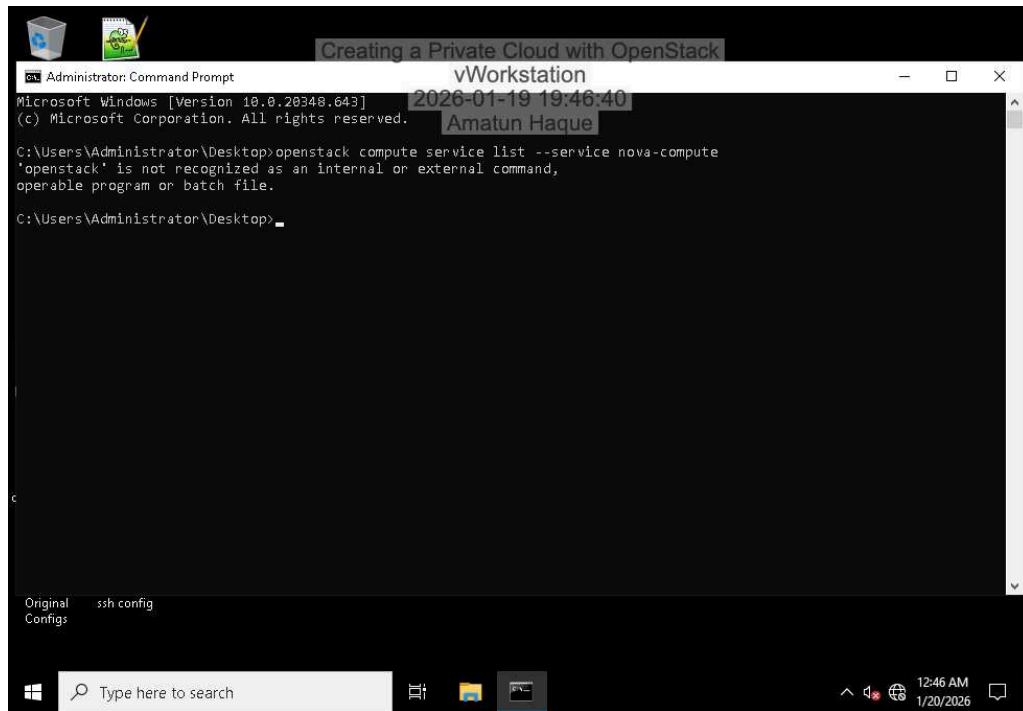
The screenshot shows a Windows desktop environment. In the background, there is a window titled "Creating a Private Cloud with OpenStack" and another titled "vWorkstation". The foreground features a terminal window titled "user@cloud1: ~" with the following content:

```
user@cloud1:~$ sudo -s /bin/sh -c "neutron-db-manage --config-file /etc/neutron/neutron.conf
--config-file /etc/neutron/plugins/m12/m12_conf/ini upgrade head" neutron
usage: neutron-db-manage [-h] [--config-dir DIR] [--config-file PATH]
                        [--subproject SUBPROJECT]
                        [--database-connection DATABASE_CONNECTION]
                        [--database-engine DATABASE_ENGINE]
                        {current,history,branches,heads,check_migration,upgrade,downgrade,stamp,revision,has_offline_migrations}
                        ...
neutron-db-manage: error: argument command: invalid choice: '/etc/neutron/plugins/m12/m12_conf/ini' (choose from 'current', 'history', 'branches', 'heads', 'check_migration', 'upgrade', 'downgrade', 'stamp', 'revision', 'has_offline_migrations')
user@cloud1:~$ sudo systemctl restart nova-api
user@cloud1:~$ sudo systemctl enable --now neutron-server
Synchronizing state of neutron-server.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable neutron-server
Created symlink /etc/systemd/system/multi-user.target.wants/neutron-server.service → /lib/systemd/system/neutron-server.service.
user@cloud1:~$ systemctl list-units --type=service "neutron*"
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
neutron-server.service             loaded active running OpenStack Neutron Server

LOAD = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB = The low-level unit activation state, values depend on unit type.
1 loaded units listed. Pass --all to see loaded but inactive units, too.
To show all installed unit files use 'systemctl list-unit-files'.
user@cloud1:~$ openstack compute service list --service nova-compute
```

The desktop taskbar at the bottom shows the Windows logo, a search bar with the text "Type here to search", and the system clock displaying "12:37 AM 1/20/2026".

31. Make a screen capture of the horizon login page.



Challenge and Analysis

Part 1: Set Up a Flavor for Nova Compute Instances

Make a screen capture showing your new flavor in the flavor list output.

Incomplete

Part 2: Add an Additional Nova Compute Resource

Make a screen capture showing both cloud2 and cloud3 in the list of available computing nodes.

Incomplete

Designing a Secure Cloud Architecture

Cloud Computing, Second Edition - Lab 04

Student:

Amatun Haque

Email:

amatunh.student@wust.edu

Time on Task:

14 hours, 18 minutes

Progress:

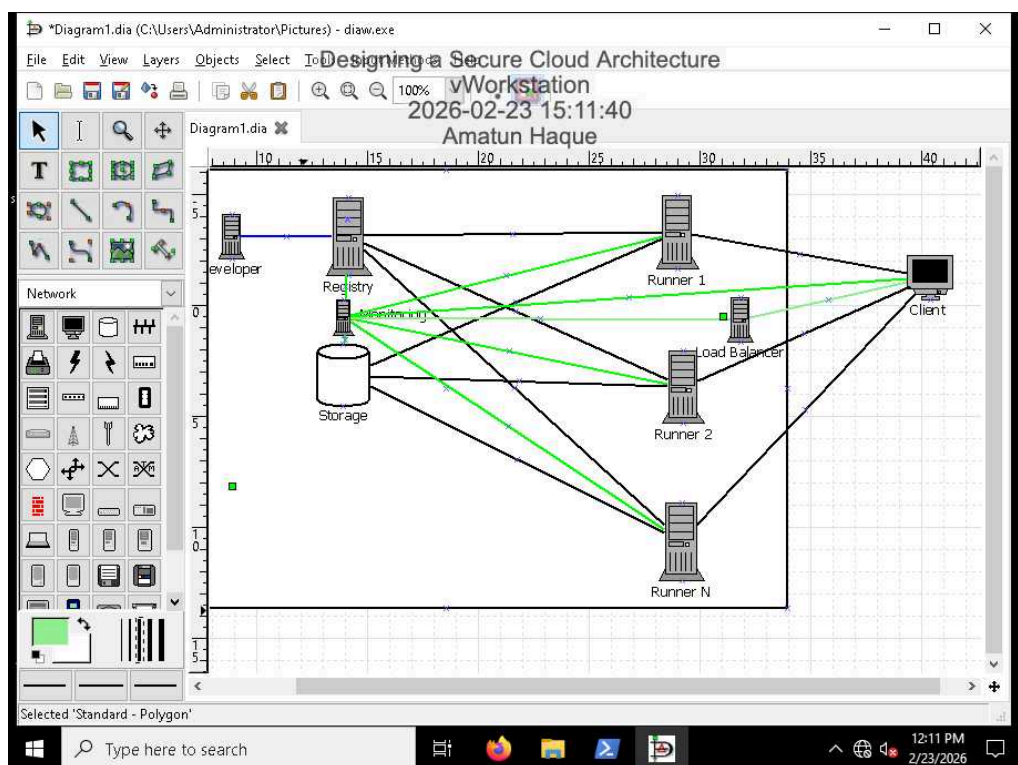
100%

Report Generated: Monday, February 23, 2026 at 4:26 PM

Hands-On Demonstration

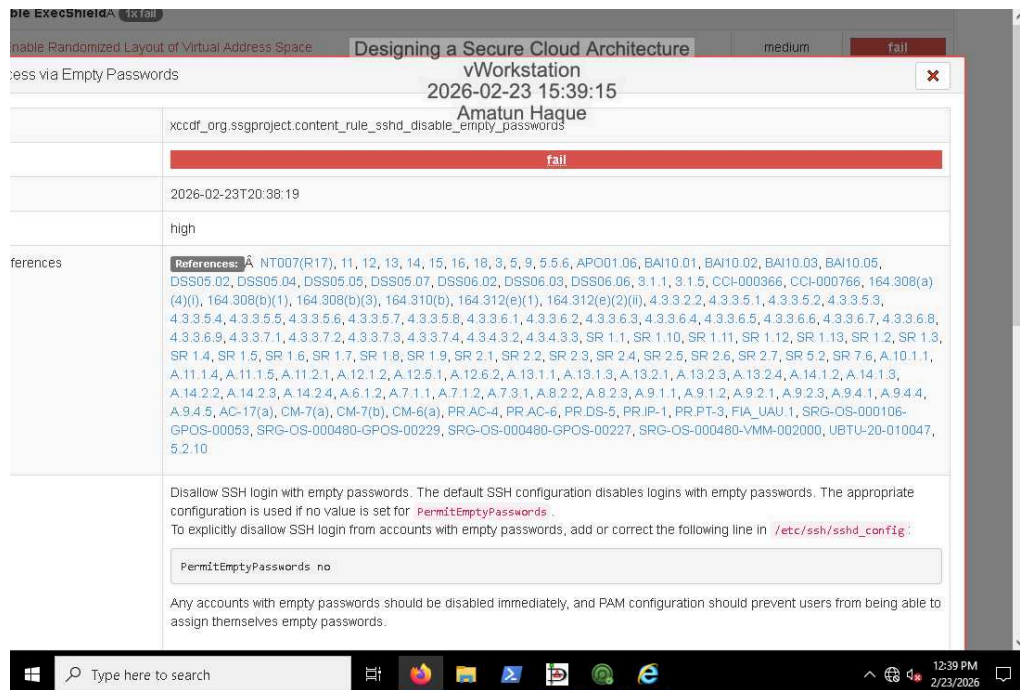
Part 1: Planning a Design

65. **Make a screen capture** showing the network diagram for the system with the labels.

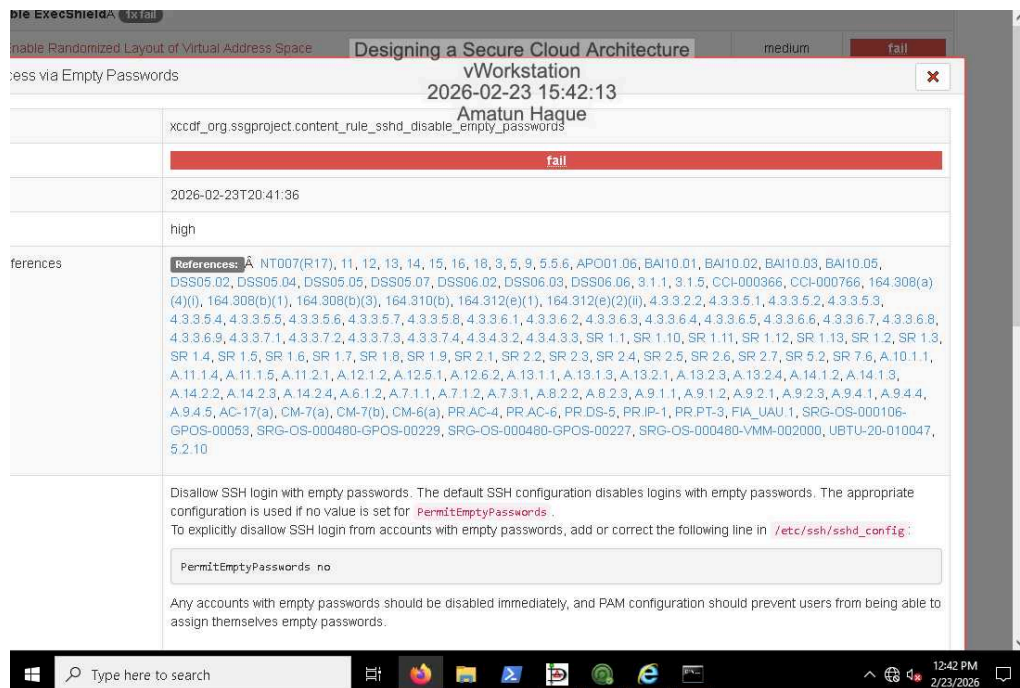


Part 2: Automating Security

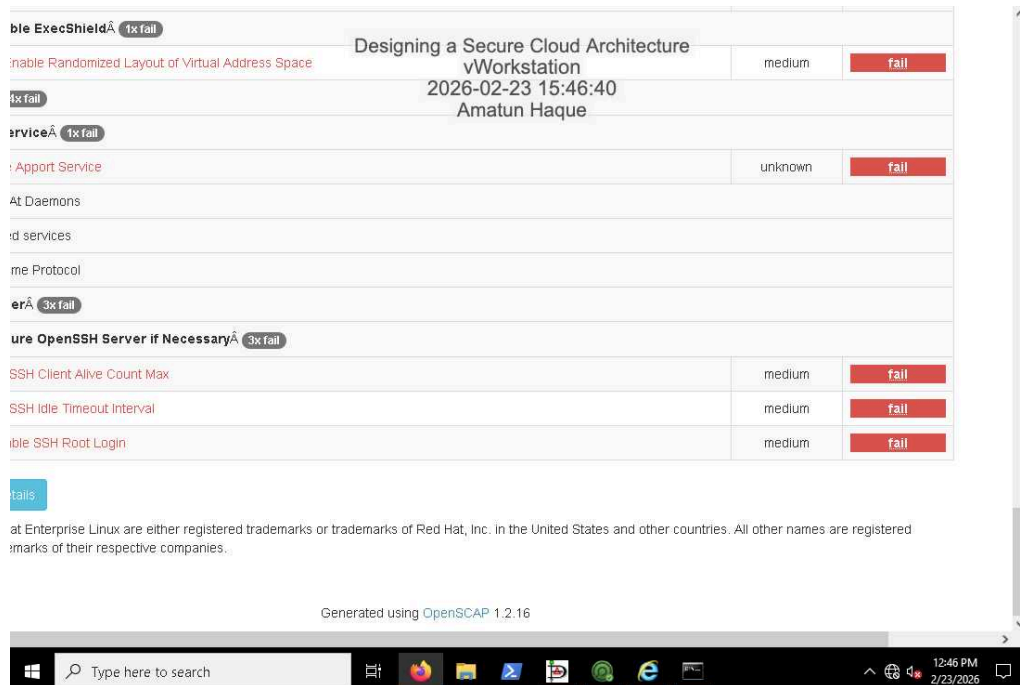
13. Make a screen capture showing the **OVAL** details of the PermitEmptyPasswords failure from the manual scan.



26. Make a screen capture showing that the Disable SSH Access via Empty Passwords test passed.

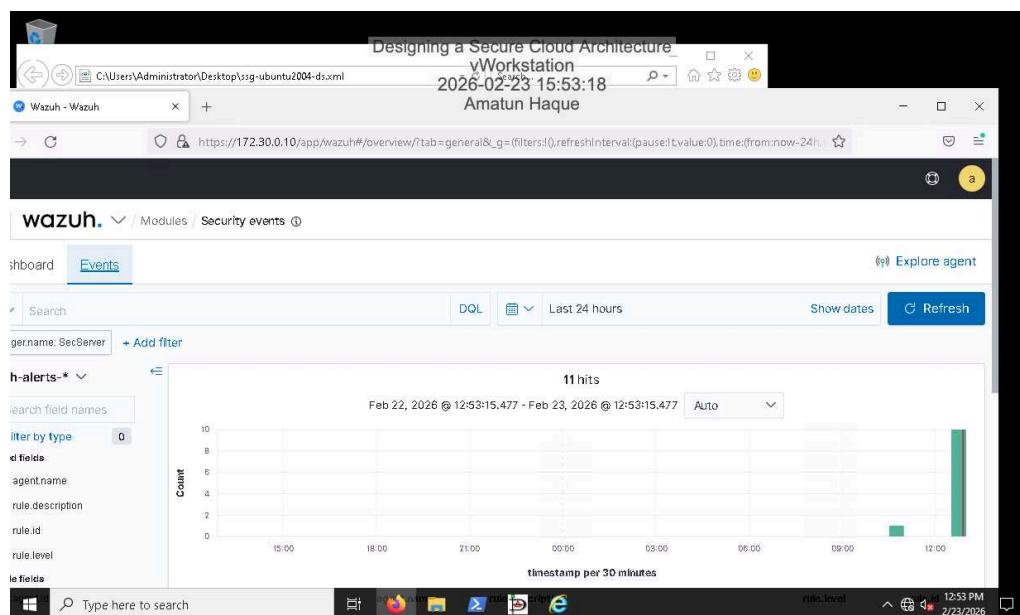


35. Make a screen capture showing the **three tests run under Services > SSH Server > Configure OpenSSH Server if Necessary**.

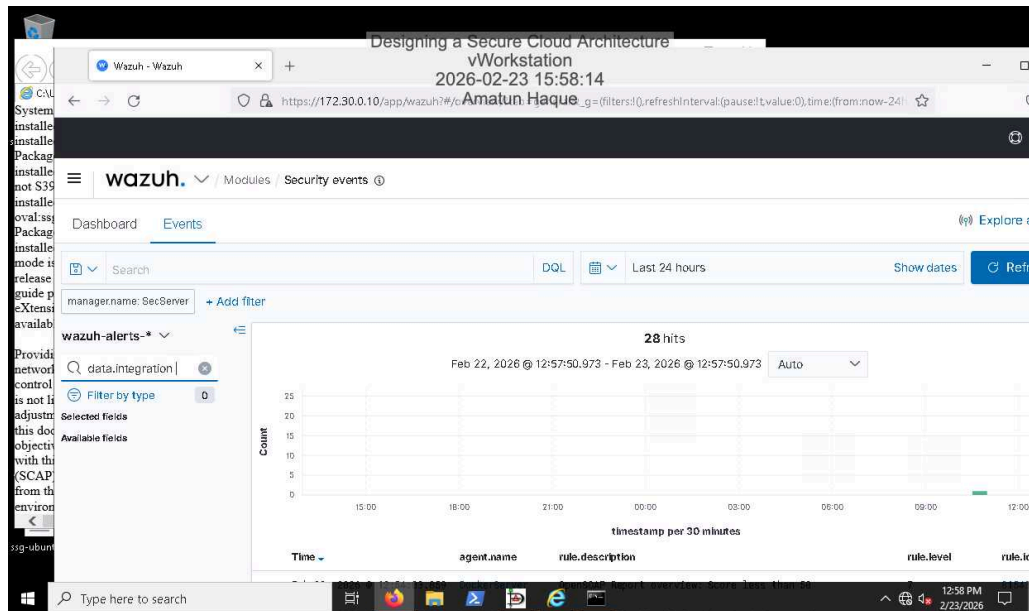


Part 3: Continuous Monitoring

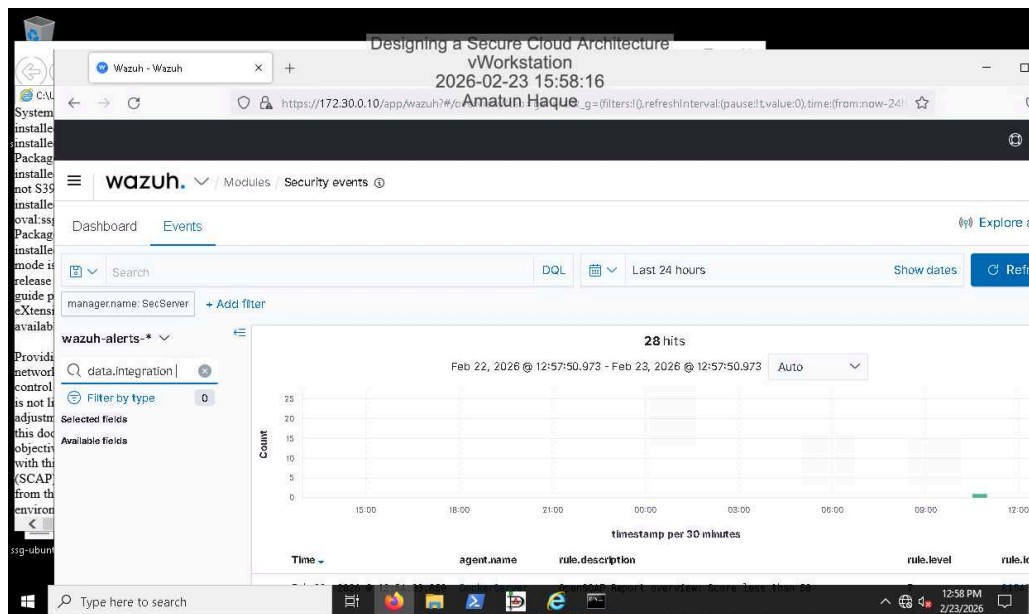
19. Make a screen capture showing an event in Wazuh for Disable SSH Access via Empty Passwords failing on the DockerRunner machine.



31. Make a screen capture showing an event about the alpine image being pulled.



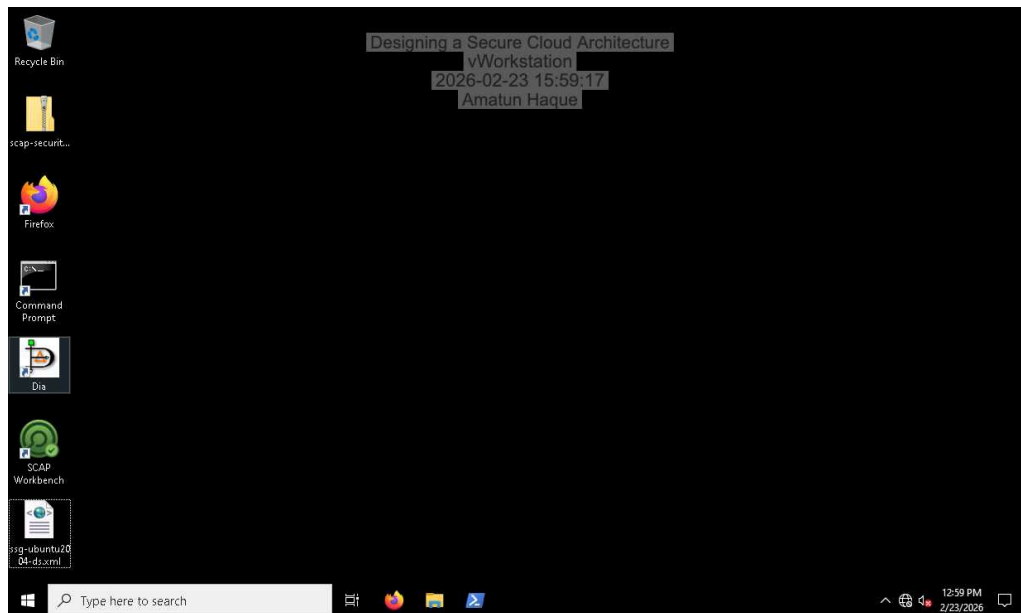
32. Make a screen capture showing an event about a container being started.



Challenge and Analysis

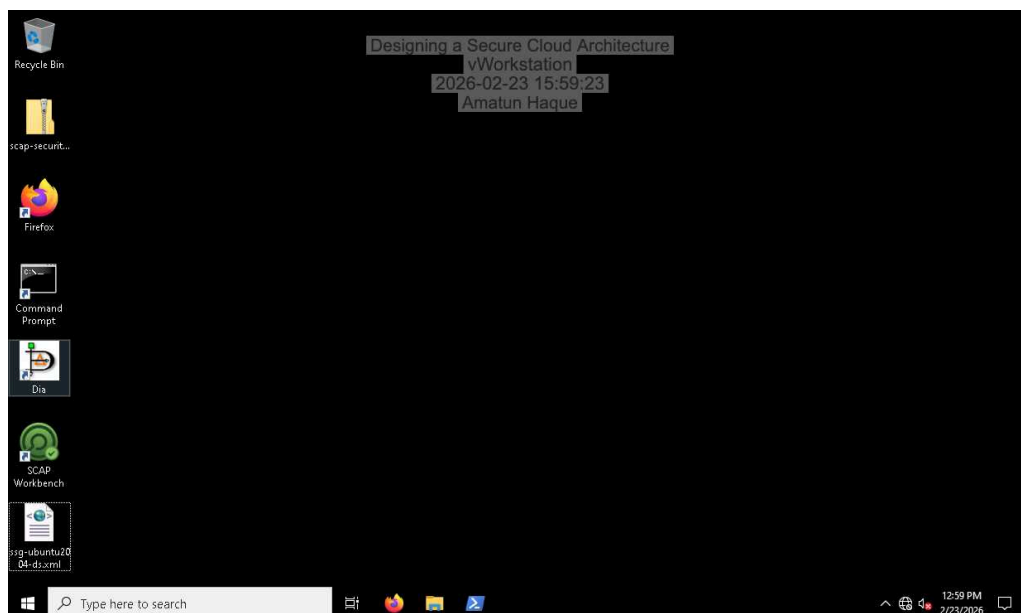
Part 1: Make a New SCAP Profile

Make a screen capture showing the results in OpenSCAP Workbench after using your new profile to scan DockerRunner.



Part 2: Update the Group Files to Use the New Profile

Make a screen capture showing the new group configuration file for the default group in Wazuh.



Student:

Amatun Haque

Email:

amatunh.student@wust.edu

Time on Task:

14 hours, 30 minutes

Progress:

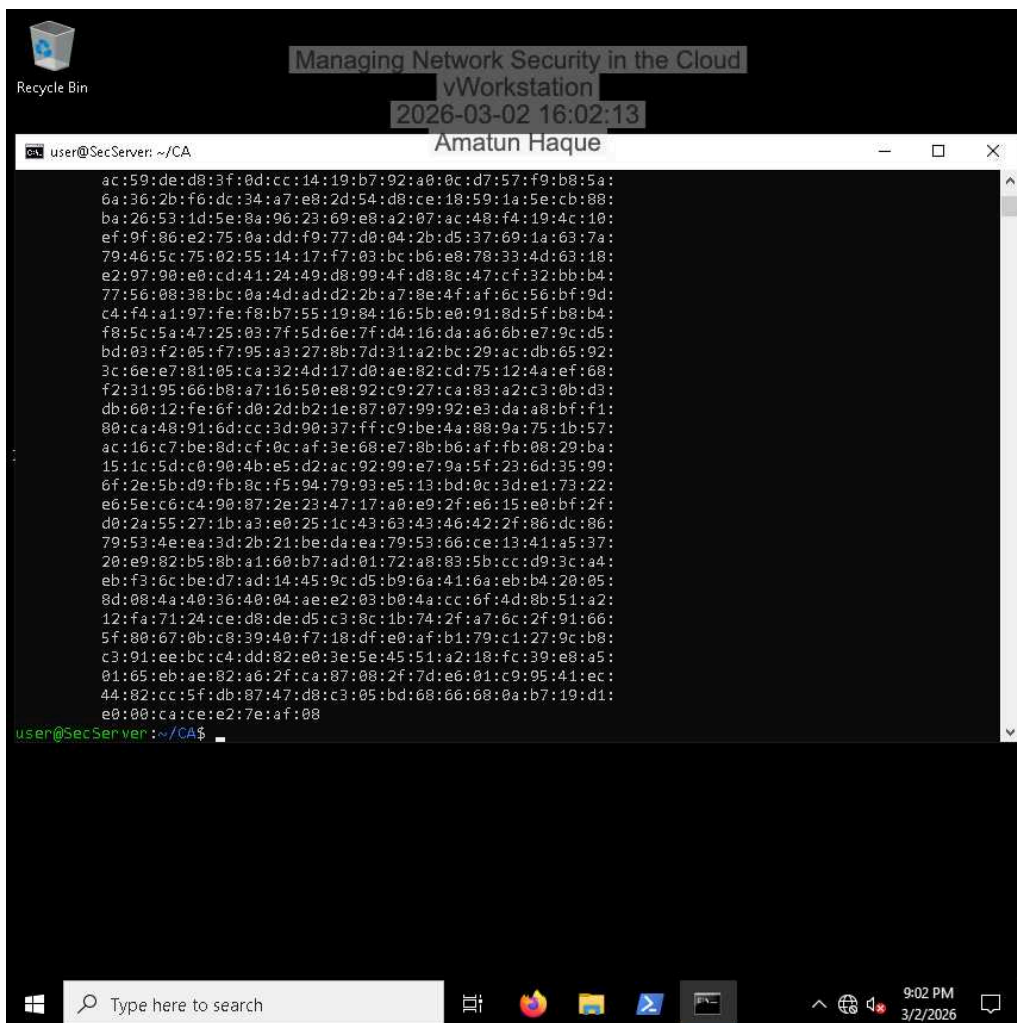
100%

Report Generated: Monday, March 2, 2026 at 5:28 PM

Hands-On Demonstration

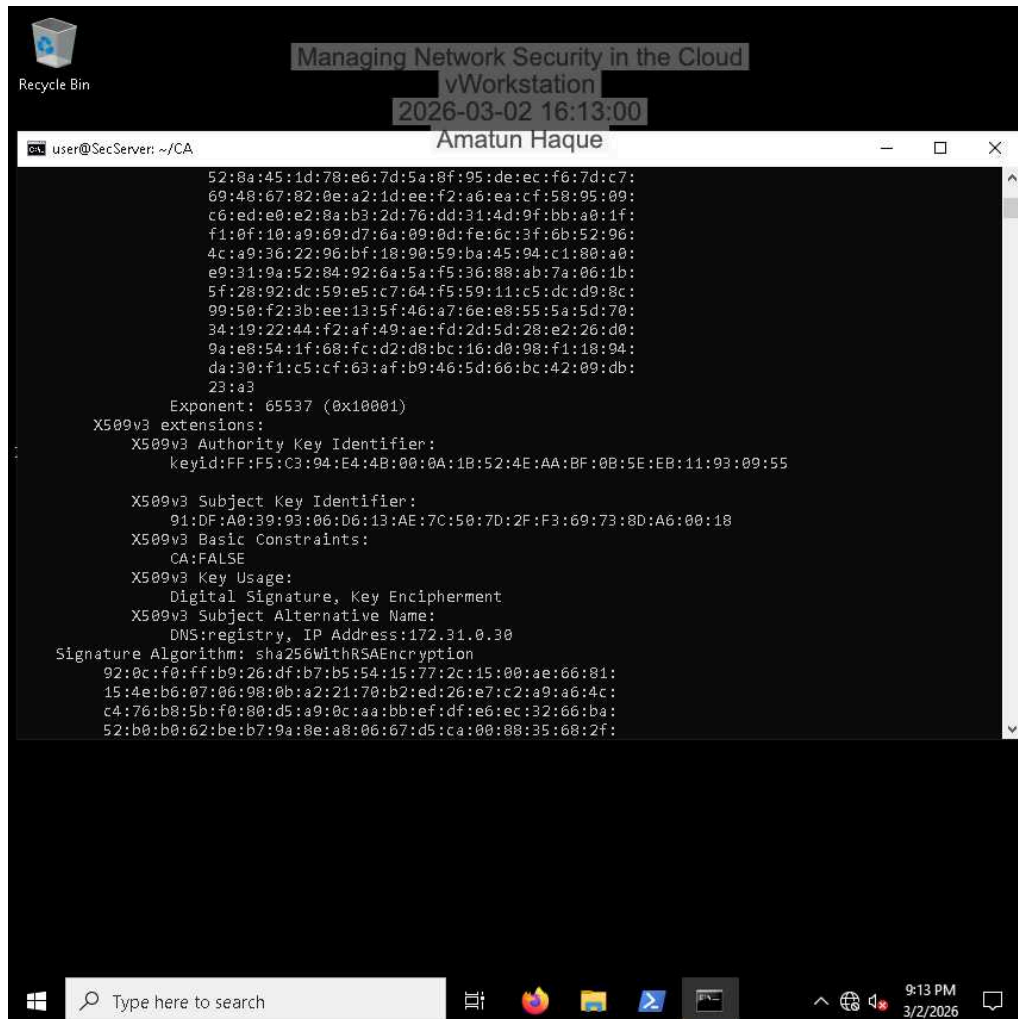
Part 1: Create Certificates for Using Mutual TLS

9. Make a screen capture showing at least the first 12 lines of the output for the root certificate.



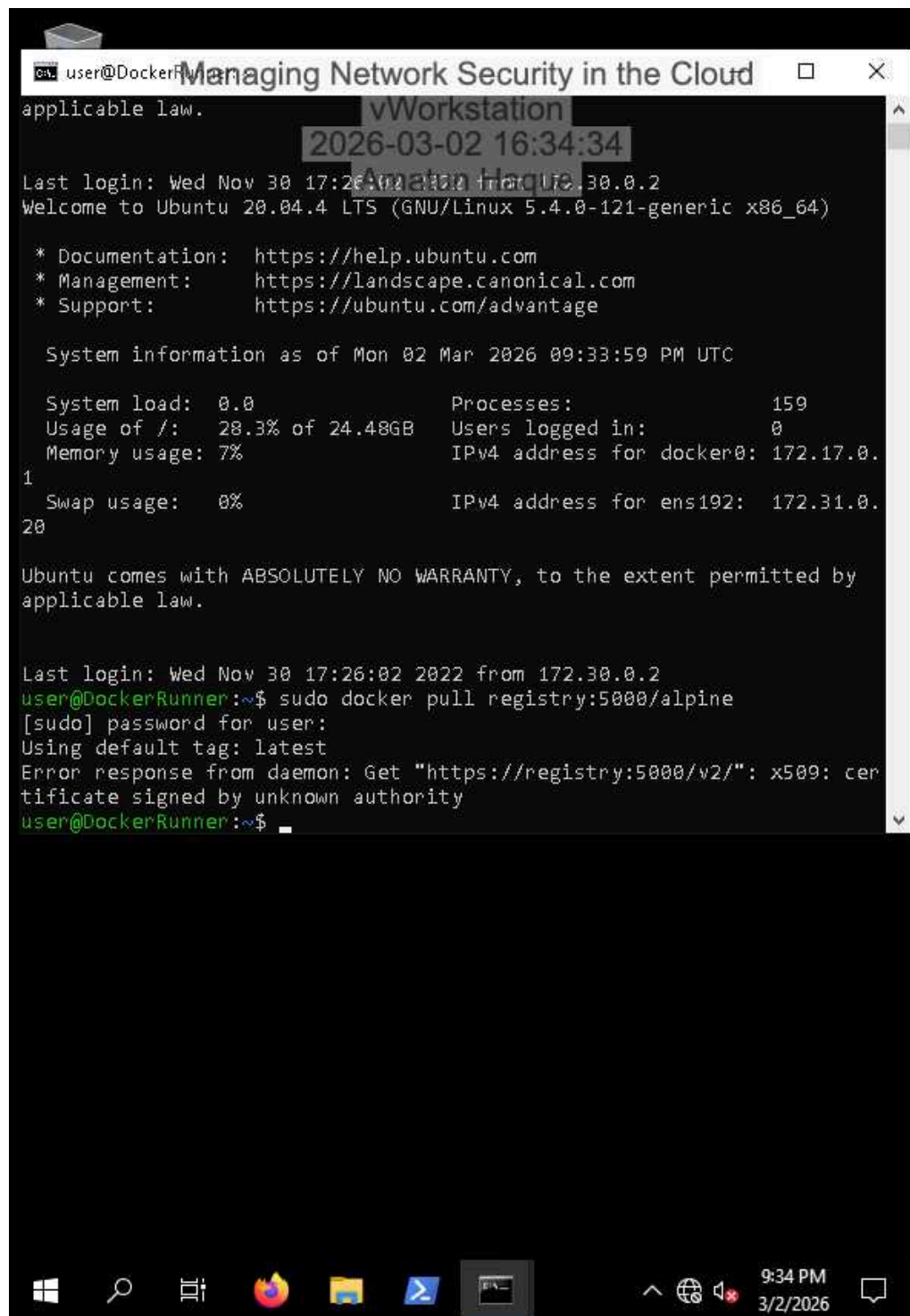
```
Managing Network Security in the Cloud
vWorkstation
2026-03-02 16:02:13
Amatun Haque
user@SecServer: ~/CA
ac:59:de:d8:3f:0d:cc:14:19:b7:92:a0:0c:d7:57:f9:b8:5a:
6a:36:2b:f6:dc:34:a7:e8:2d:54:d8:ce:18:59:1a:5e:cb:88:
ba:26:53:1d:5e:8a:96:23:69:e8:a2:07:ac:48:f4:19:4c:10:
ef:9f:86:e2:75:0a:dd:f9:77:d0:04:2b:d5:37:69:1a:63:7a:
79:46:5c:75:02:55:14:17:f7:03:bc:b6:e8:78:33:4d:63:18:
e2:97:90:e0:cd:41:24:49:d8:99:4f:d8:8c:47:cf:32:bb:b4:
77:56:08:38:bc:0a:4d:ad:d2:2b:a7:8e:4f:af:6c:56:bf:9d:
c4:f4:a1:97:fe:f8:b7:55:19:84:16:5b:e0:91:8d:5f:b8:b4:
f8:5c:5a:47:25:03:7f:5d:6e:7f:d4:16:da:a6:6b:e7:9c:d5:
bd:03:f2:05:f7:95:a3:27:8b:7d:31:a2:bc:29:ac:db:65:92:
3c:6e:e7:81:05:ca:32:4d:17:d0:ae:82:cd:75:12:4a:ef:68:
f2:31:95:66:b8:a7:16:50:e8:92:c9:27:ca:83:a2:c3:0b:d3:
db:60:12:fe:6f:d0:2d:b2:1e:87:07:99:92:e3:da:a8:bf:f1:
80:ca:48:91:6d:cc:3d:90:37:ff:c9:be:4a:88:9a:75:1b:57:
ac:16:c7:be:8d:cf:0c:af:3e:68:e7:8b:b6:af:fb:08:29:ba:
15:1c:5d:c0:90:4b:e5:d2:ac:92:99:e7:9a:5f:23:6d:35:99:
6f:2e:5b:d9:fb:8c:f5:94:79:93:e5:13:bd:0c:3d:e1:73:22:
e6:5e:c6:c4:90:87:2e:23:47:17:a0:e9:2f:e6:15:e0:bf:2f:
d0:2a:55:27:1b:a3:e0:25:1c:43:63:43:46:42:2f:86:dc:86:
79:53:4e:ea:3d:2b:21:be:da:ea:79:53:66:ce:13:41:a5:37:
20:e9:82:b5:8b:a1:60:b7:ad:01:72:a8:83:5b:cc:d9:3c:a4:
eb:f3:6c:be:d7:ad:14:45:9c:d5:b9:6a:41:6a:eb:b4:20:05:
8d:08:4a:40:36:40:04:ae:e2:03:b0:4a:cc:6f:4d:8b:51:a2:
12:fa:71:24:ce:d8:de:d5:c3:8c:1b:74:2f:a7:6c:2f:91:66:
5f:80:67:0b:c0:39:40:f7:18:df:e0:af:b1:70:c1:27:9c:b8:
c3:91:ee:bc:c4:dd:82:e0:3e:5e:45:51:a2:18:fc:39:e8:a5:
01:65:eb:ae:82:a6:2f:ca:87:08:2f:7d:e6:01:c9:95:41:ec:
44:82:cc:5f:db:87:47:d8:c3:05:bd:68:66:68:0a:b7:19:d1:
e0:00:ca:ce:e2:7e:af:08
user@SecServer:~/CA$
```

15. Make a screen capture showing the X509v3 extensions of the registry certificate.



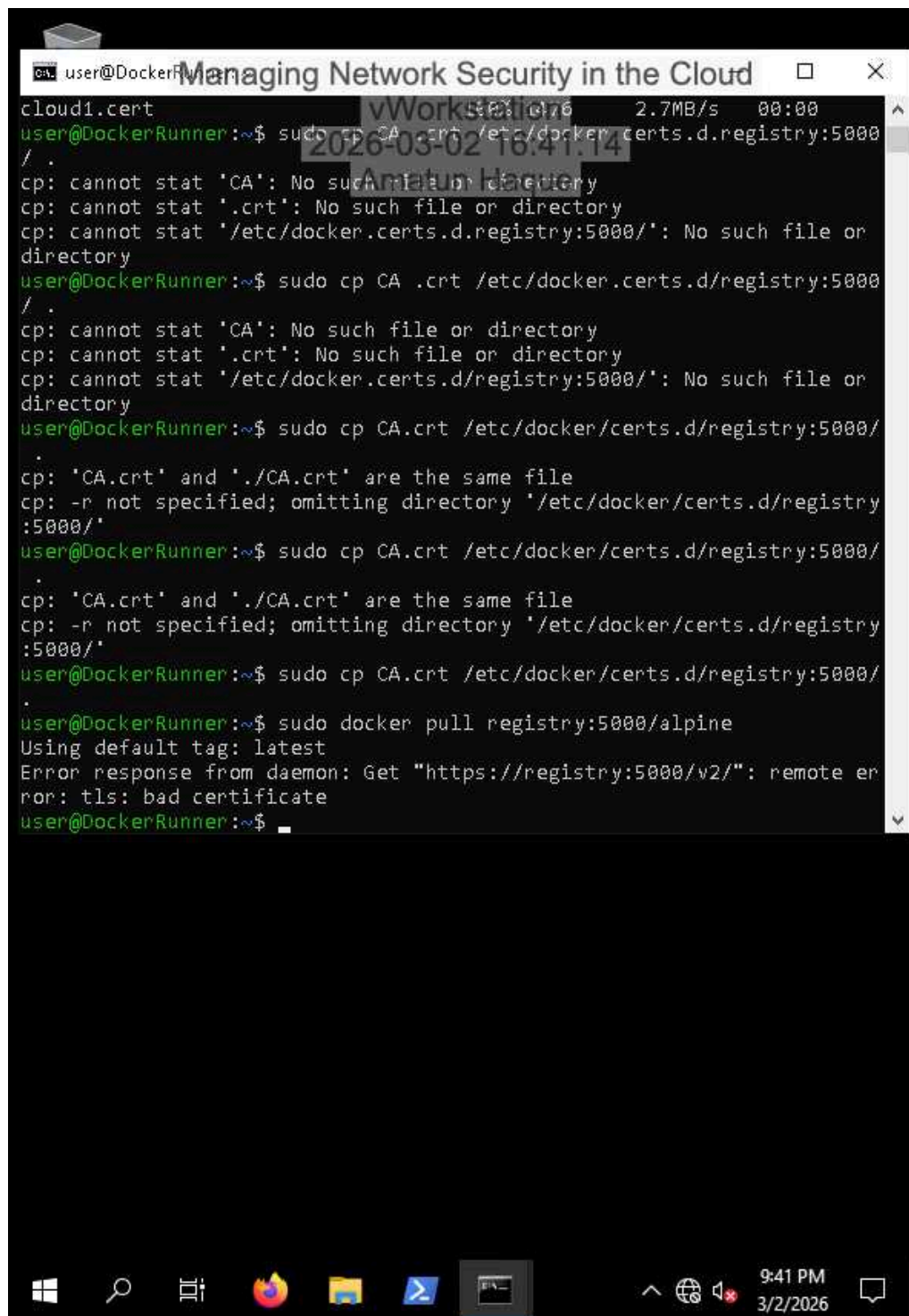
Part 2: Enable Zero Trust for Docker

16. Make a screen capture showing the unknown certificate authority error.



```
user@DockerRunner: ~$  
Last login: Wed Nov 30 17:26:02 2022 from 172.30.0.2  
Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-121-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/advantage  
  
System information as of Mon 02 Mar 2026 09:33:59 PM UTC  
  
System load:  0.0          Processes:            159  
Usage of /:   28.3% of 24.48GB Users logged in:      0  
Memory usage: 7%          IPv4 address for docker0: 172.17.0.1  
Swap usage:   0%          IPv4 address for ens192: 172.31.0.20  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
Last login: Wed Nov 30 17:26:02 2022 from 172.30.0.2  
user@DockerRunner:~$ sudo docker pull registry:5000/alpine  
[sudo] password for user:  
Using default tag: latest  
Error response from daemon: Get "https://registry:5000/v2/": x509: cer  
tificate signed by unknown authority  
user@DockerRunner:~$
```

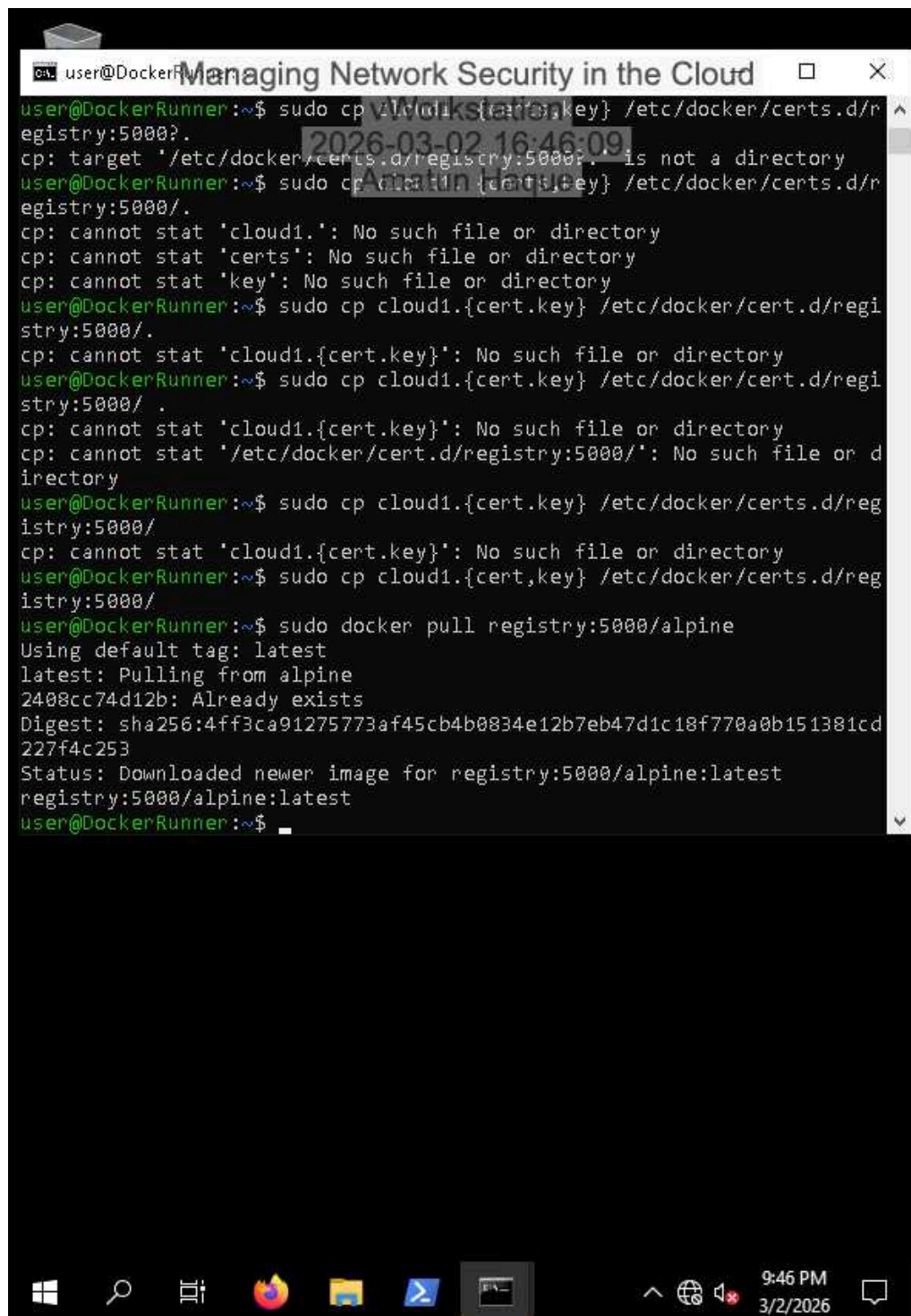
21. Make a screen capture showing the bad certificate error.



The screenshot shows a terminal window titled "Managing Network Security in the Cloud" with a user@DockerRunner prompt. The terminal displays several failed attempts to copy a file named "CA.crt" to a directory "/etc/docker/certs.d/registry:5000/". The errors indicate that the source file does not exist or that the paths are the same. Finally, the user runs the command "sudo docker pull registry:5000/alpine", which results in an error: "Error response from daemon: Get \"https://registry:5000/v2/\": remote error: tls: bad certificate". The terminal window is overlaid on a desktop environment with a taskbar at the bottom showing various application icons and the system clock indicating 9:41 PM on 3/2/2026.

```
cloud1.cert
user@DockerRunner:~$ sudo cp CA.crt /etc/docker/certs.d/registry:5000/
cp: cannot stat 'CA': No such file or directory
cp: cannot stat '.crt': No such file or directory
cp: cannot stat '/etc/docker/certs.d/registry:5000/': No such file or directory
user@DockerRunner:~$ sudo cp CA .crt /etc/docker/certs.d/registry:5000/
cp: cannot stat 'CA': No such file or directory
cp: cannot stat '.crt': No such file or directory
cp: cannot stat '/etc/docker/certs.d/registry:5000/': No such file or directory
user@DockerRunner:~$ sudo cp CA.crt /etc/docker/certs.d/registry:5000/
cp: 'CA.crt' and './CA.crt' are the same file
cp: -r not specified; omitting directory '/etc/docker/certs.d/registry:5000/'
user@DockerRunner:~$ sudo cp CA.crt /etc/docker/certs.d/registry:5000/
cp: 'CA.crt' and './CA.crt' are the same file
cp: -r not specified; omitting directory '/etc/docker/certs.d/registry:5000/'
user@DockerRunner:~$ sudo cp CA.crt /etc/docker/certs.d/registry:5000/
user@DockerRunner:~$ sudo docker pull registry:5000/alpine
Using default tag: latest
Error response from daemon: Get "https://registry:5000/v2/": remote error: tls: bad certificate
user@DockerRunner:~$
```

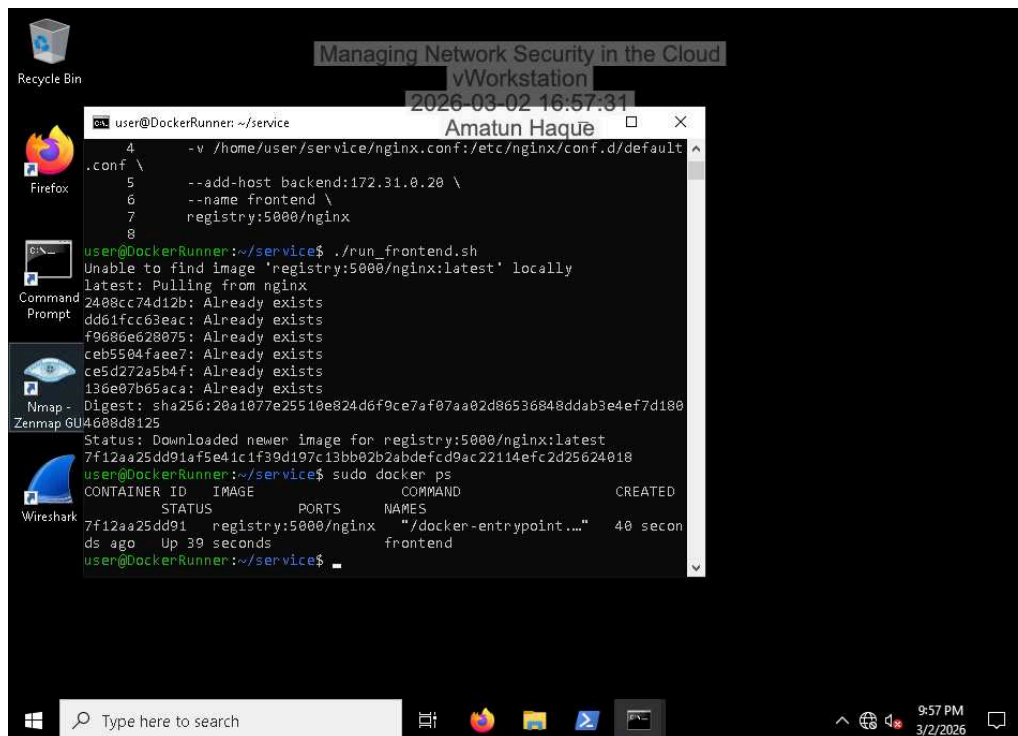
24. **Make a screen capture** showing a successful pull of registry:5000/alpine on DockerRunner.



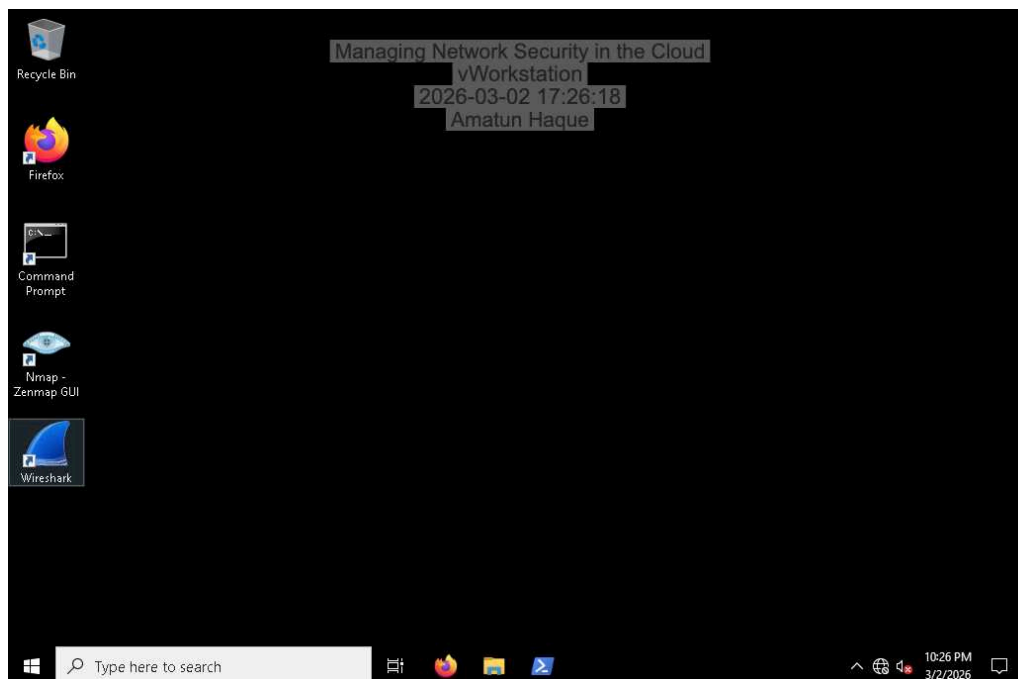
```
user@DockerRunner:~$ sudo cp cloud1.{cert,key} /etc/docker/certs.d/registry:5000/.
cp: target '/etc/docker/certs.d/registry:5000/' is not a directory
user@DockerRunner:~$ sudo cp cloud1.{cert,key} /etc/docker/certs.d/registry:5000/.
cp: cannot stat 'cloud1.': No such file or directory
cp: cannot stat 'certs': No such file or directory
cp: cannot stat 'key': No such file or directory
user@DockerRunner:~$ sudo cp cloud1.{cert.key} /etc/docker/certs.d/registry:5000/.
cp: cannot stat 'cloud1.{cert.key}': No such file or directory
user@DockerRunner:~$ sudo cp cloud1.{cert.key} /etc/docker/certs.d/registry:5000/.
cp: cannot stat 'cloud1.{cert.key}': No such file or directory
cp: cannot stat '/etc/docker/certs.d/registry:5000/': No such file or directory
user@DockerRunner:~$ sudo cp cloud1.{cert.key} /etc/docker/certs.d/registry:5000/.
cp: cannot stat 'cloud1.{cert.key}': No such file or directory
user@DockerRunner:~$ sudo cp cloud1.{cert,key} /etc/docker/certs.d/registry:5000/.
user@DockerRunner:~$ sudo docker pull registry:5000/alpine
Using default tag: latest
latest: Pulling from alpine
2408cc74d12b: Already exists
Digest: sha256:4ff3ca91275773af45cb4b0834e12b7eb47d1c18f770a0b151381cd227f4c253
Status: Downloaded newer image for registry:5000/alpine:latest
registry:5000/alpine:latest
user@DockerRunner:~$
```

Part 3: Using Zero Trust for a Web Service

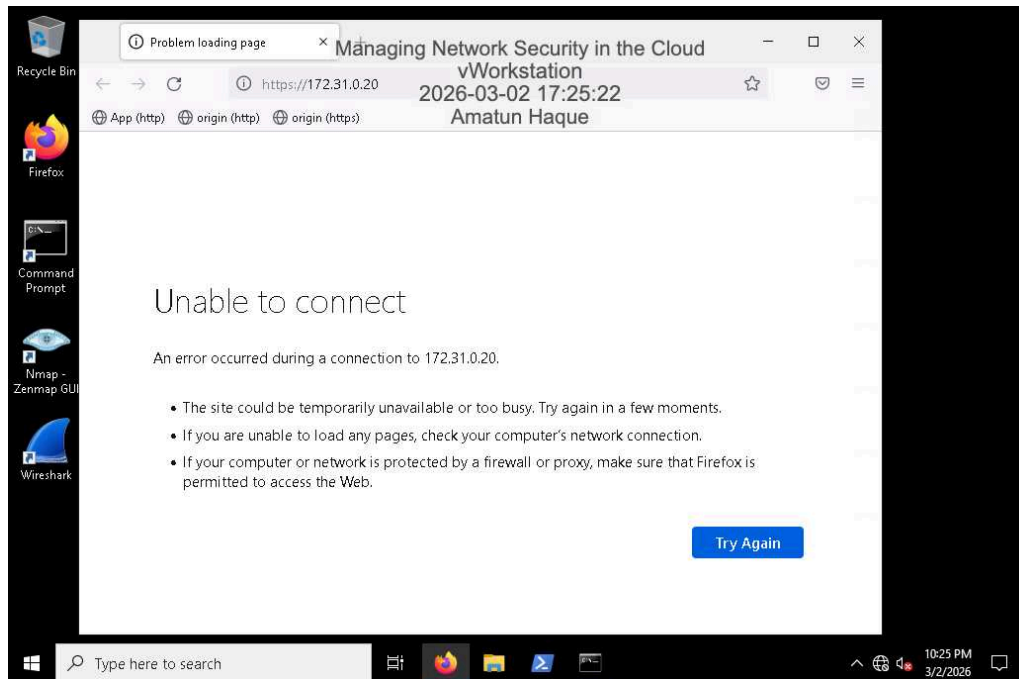
13. **Make a screen capture** showing the running frontend and backend containers.



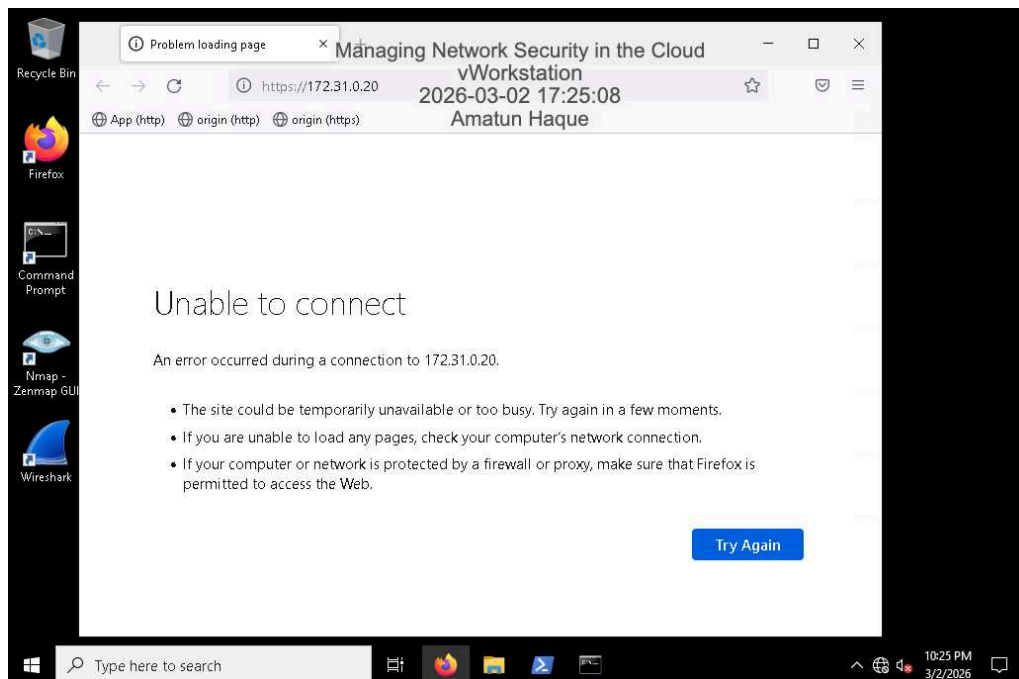
16. **Make a screen capture** showing the successful page load from App (http).



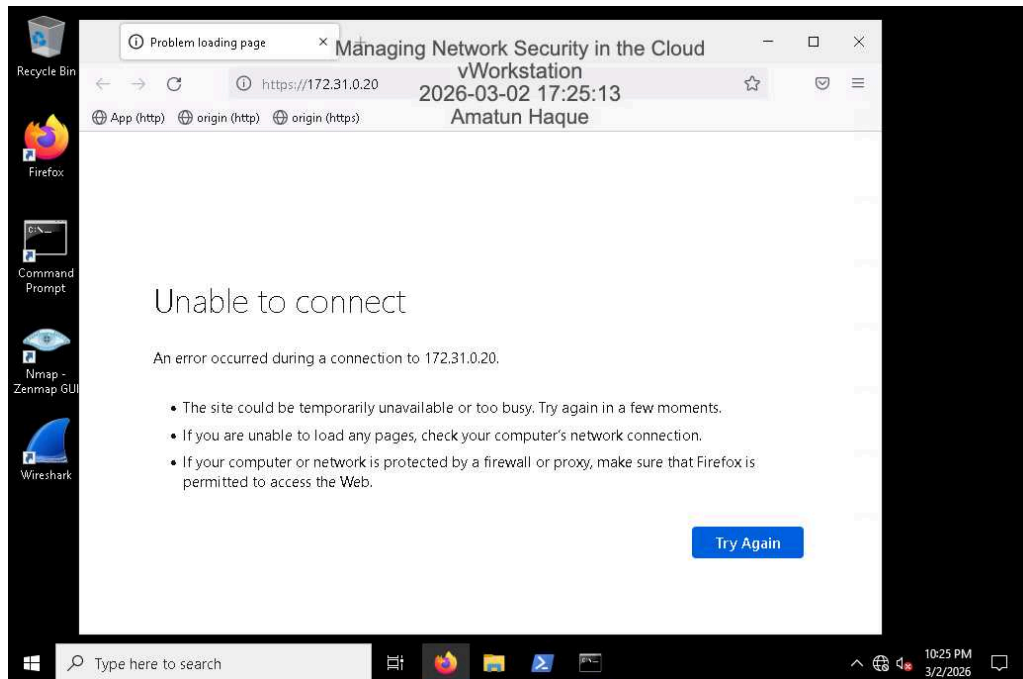
18. Make a screen capture showing the successful page load from origin (http).



30. Make a screen capture showing the successful page load from App (http) with mutual TLS on.



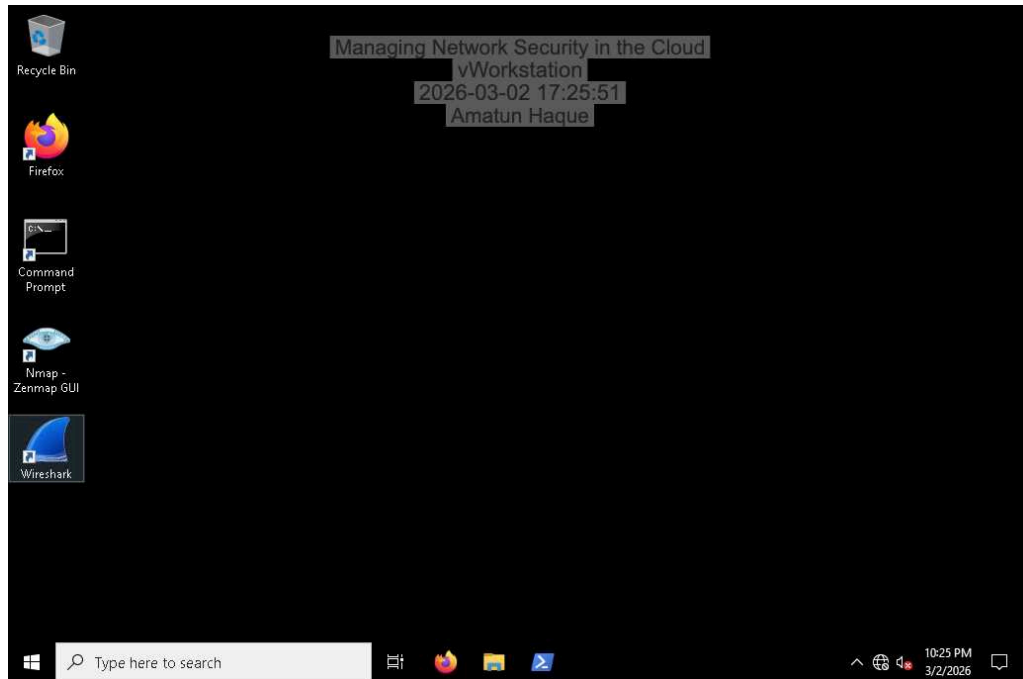
33. **Make a screen capture** showing the unsuccessful load from origin (https).



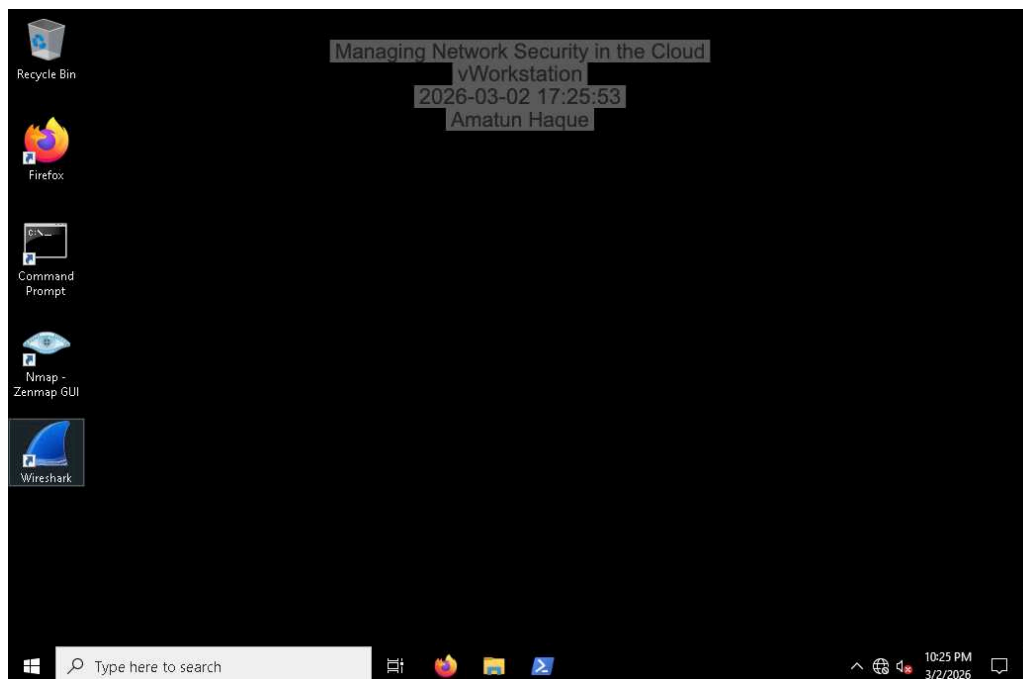
Challenge and Analysis

Part 1: Enable Mutual TLS for the Development Machine

1. **Make a screen capture** showing at least the first 12 lines of the client certificate



2. **Make a screen capture** showing a successful pull of registry:5000/alpine on SecServer.



Part 2: Check Certificates Manually

1. **Make a screen capture** showing the output of the verification command for the developer certificate.

